

InnoDB Architecture Deep-Dive

wilhasse/courses

InnoDB Architecture Deep-Dive

Start here

Chapters

Continue the journey

Why this codebase for studying MySQL's early days

Chapter 0 — Overview: How InnoDB Is Architected

Why study this codebase?

The big picture

The layered mental model

Chapter roadmap

How to study with this series

Conventions used in this series

Chapter 1 — Files, Tablespaces & Space Management

1.1 The unit of everything: the page

1.2 The FIL layer: mapping spaces to real files

1.3 The FSP layer: allocating pages inside a space

1.4 What to remember

Chapter 2 — Inside the 16KB Page & the Record Format

2.1 Every page starts and ends the same way

2.2 The index page: a heap of records plus a sorted directory

2.3 The record format: how a row is encoded

2.4 What to remember

Chapter 3 — The Buffer Pool

3.1 Why a buffer pool

3.2 The data structures

3.3 Fetching a page: `buf_page_get_gen()`

3.4 Writing pages back: the flush list

3.5 What to remember

Chapter 4 — Mini-Transactions & Latching

4.1 The problem mini-transactions solve

4.2 Anatomy and lifecycle

4.3 Redo records: physiological logging

4.4 Latching: rw-locks and the ordering discipline

4.5 What to remember

Chapter 5 — The Redo Log & Crash Recovery

5.1 The LSN: one number to rule them all

- 5.2 The log on disk: blocks, files, checkpoint slots
- 5.3 Runtime: buffer, group commit, and the WAL rule
- 5.4 Checkpoints: bounding recovery and reclaiming log space
- 5.5 Crash recovery
- 5.6 What to remember

Chapter 6 — The B+Tree

- 6.1 Everything is a B+tree
- 6.2 Searching: latch crabbing with a tree latch
- 6.3 Inserting: optimistic first, pessimistic if needed
- 6.4 The adaptive hash index
- 6.5 What to remember

Chapter 7 — Transactions, Undo Logs & MVCC

- 7.1 The transaction object and system
- 7.2 Undo logs: the before-image store
- 7.3 Read views: deciding what a transaction may see
- 7.4 Commit and rollback
- 7.5 Purge: taking out the MVCC garbage
- 7.6 What to remember

Chapter 8 — The Lock Manager

- 8.1 Locks vs latches — don't confuse them
- 8.2 Modes and intention locking
- 8.3 Record locks: bitmaps and gap semantics
- 8.4 Waiting, timeouts, deadlocks
- 8.5 What to remember

Chapter 9 — Row Operations: INSERT, SELECT, UPDATE, DELETE End-to-End

- 9.1 The cast of characters
- 9.2 INSERT
- 9.3 SELECT
- 9.4 UPDATE and DELETE
- 9.5 The other half: UNDO and PURGE as row operations
- 9.6 Bonus: fast index creation
- 9.7 What to remember

Chapter 10 — The Data Dictionary & the Internal SQL Interpreter

- 10.1 The bootstrapping trick
- 10.2 The in-memory cache
- 10.3 A SQL interpreter inside a storage engine?
- 10.4 What to remember

Chapter 11 — Startup, Shutdown & the Embedded API

- 11.1 The embedded API in one test program
- 11.2 What `ib_startup()` actually does
- 11.3 Configuration: the `srv_*` globals
- 11.4 API call → internal machinery

11.5 What to remember

Chapter 12 — Background Threads, the Master Thread & the Insert Buffer

12.1 The thread roster

12.2 The master thread: the engine's heartbeat

12.3 The insert buffer: turning random I/O into sequential

12.4 The full picture

12.5 What to remember

InnoDB Architecture Deep-Dive

A 13-chapter, step-by-step study guide to how InnoDB was architected — from the bytes on disk up to ACID transactions — based on the source code of **Embedded InnoDB 1.0.6** (the Innobase Oy era, before deep MySQL integration). Every claim is grounded in real source with `file.c:line` references, plus Mermaid diagrams and byte-layout tables.

Source repository studied: [wilhasse/oss-embedded-innodb](https://github.com/wilhasse/oss-embedded-innodb) — all file references in the chapters (e.g. `btr/btr0cur.c:345`) point into that tree. Clone it alongside to read the code as you go; it builds and has runnable test programs (`tests/`) for the hands-on exercises at the end of each chapter.

Start here

➔ **Chapter 0 — Overview: How InnoDB Is Architected** — the big-picture map, a five-layer mental model, and how to study with this series.

Chapters

The order is bottom-up: each layer only makes sense in terms of the one below it.

#	Chapter	Question it answers
00	Overview & Roadmap	How do all the pieces fit together?
01	Files, Tablespaces & Space Management	Where do bytes live? How are pages allocated?
02	The 16KB Page & Record Format	What exactly is inside a page? How is a row encoded?
03	The Buffer Pool	How are pages cached, evicted, and safely written back?
04	Mini-Transactions & Latching	How is a page change made atomic and logged?
05	Redo Log & Crash Recovery	How do commits survive any crash?
06	The B+Tree	How are rows indexed, found, split, merged?
07	Transactions, Undo & MVCC	How do commit, rollback, and snapshot reads work?
08	The Lock Manager	How are conflicts, phantoms, and deadlocks handled?
09	Row Operations	What happens end-to-end on INSERT/SELECT/UPDATE/DELETE?
10	The Data Dictionary	Where does InnoDB keep its own schema?
11	Startup, Shutdown & the Embedded API	What happens between <code>ib_init()</code> and a usable database?
12	Background Threads & the Insert Buffer	Who does the work transactions defer?

Continue the journey

This series covers the storage engine. The companion **MySQL Server Architecture Deep-Dive** covers everything *above* it — connections, parser, optimizer, executor, binlog, replication — based on Percona Server 8.0, and meets this series at the handler API.

Why this codebase for studying MySQL's early days

This is InnoDB as a **pure storage engine** — no MySQL parser, replication, or optimizer — in ~250k lines of readable C. Nearly every mechanism here still exists in MySQL 8.x InnoDB (`innodb_flush_log_at_trx_commit`, the midpoint LRU, the doublewrite buffer, next-key locking, history list length...), just with more layers on top. The chapters point out these connections as they appear, so the folklore of two decades of MySQL tuning maps back to the code that created it.

Chapter 0 — Overview: How InnoDB Is Architected

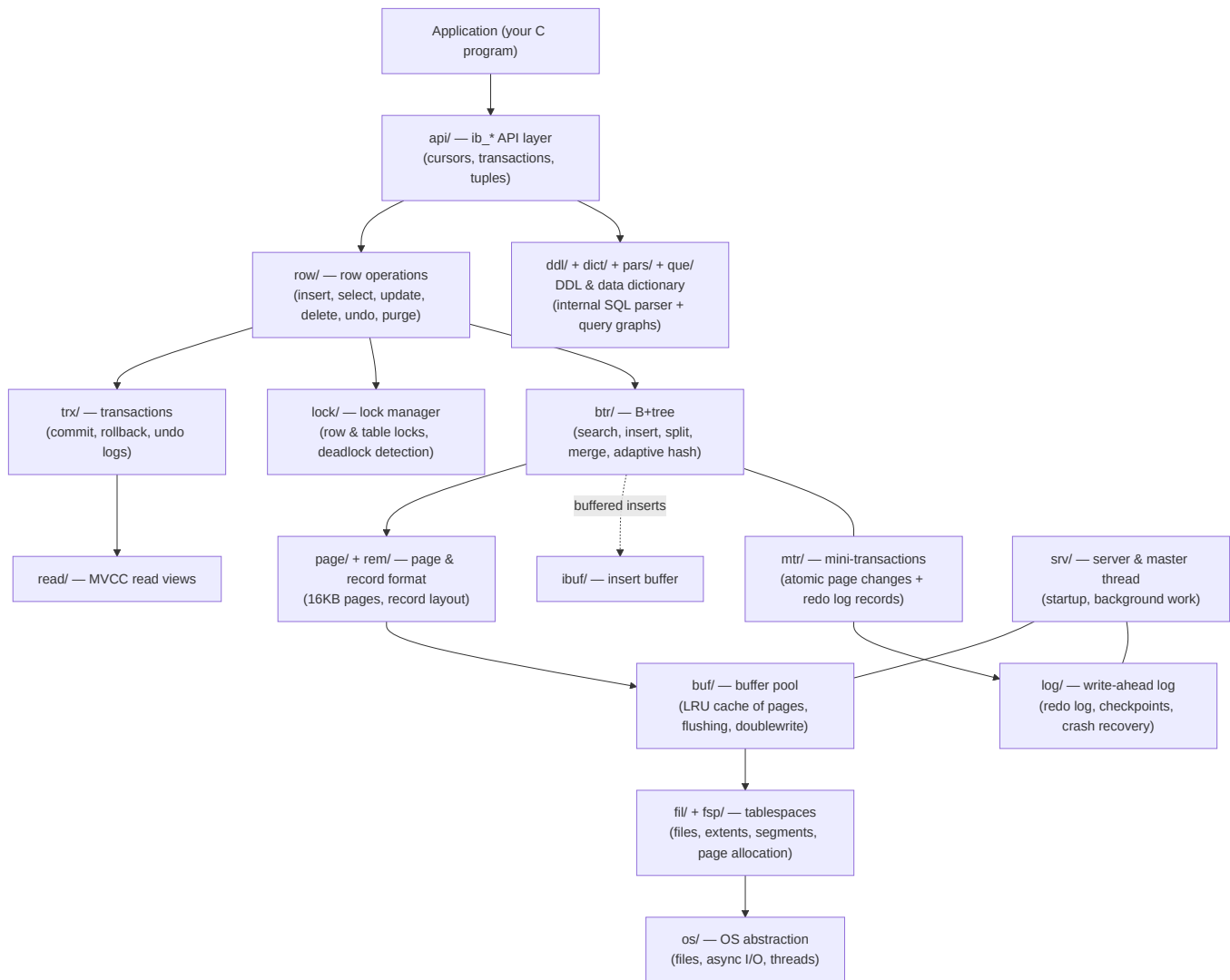
Series: InnoDB Architecture Deep-Dive — a step-by-step study guide to the internals of Embedded InnoDB 1.0.6 (Innobase Oy era, before deep MySQL integration). Every claim in this series is grounded in *this repository's* source code, with file references you can open and read alongside each chapter.

Why study this codebase?

This is InnoDB as a **pure storage engine**: no MySQL parser, no replication, no query optimizer. What remains is exactly the part that made InnoDB famous — a crash-safe, transactional, multi-versioned B+tree storage engine — in roughly 250k lines of readable C. Almost every idea here still exists in today's MySQL 8.x InnoDB, just with more layers on top. If you understand this codebase, you understand the core of modern InnoDB.

The big picture

An application links `libinnodb` and talks to the `ib_*` API. Underneath, the engine is a stack of cooperating subsystems:



Three cross-cutting mechanisms tie everything together:

1. **Mini-transactions (mtr/)** — every physical page change is wrapped in a mini-transaction that (a) holds page latches so the change is atomic, and (b) emits redo log records so the change is durable. This is the write-ahead-logging backbone.
2. **The buffer pool (buf/)** — nobody reads or writes disk pages directly; every page access goes through the buffer pool, which caches 16KB pages in memory with an LRU policy.
3. **MVCC (trx/ + read/ + undo logs)** — writers never block readers: updates keep the old row version reachable through undo logs, and each reader reconstructs the version it is allowed to see.

The layered mental model

A good way to hold the engine in your head is as five layers, from bytes on disk up to ACID transactions:

5. TRANSACTIONS	ACID, MVCC, locking	trx/ lock/ read/
4. ACCESS PATHS	B+trees, cursors, row ops	btr/ row/ ibuf/
3. DURABILITY	mini-transactions, redo log, checkpoints, crash recovery	mtr/ log/
2. CACHING	buffer pool, LRU, flushing, doublewrite, read-ahead	buf/
1. STORAGE	tablespaces, extents, segments, 16KB pages, record format	fil/ fsp/ page/ rem/ os/

The chapters follow this bottom-up order, because each layer only makes sense in terms of the one below it: you can't understand the redo log until you know what a page is, and you can't understand MVCC until you know how undo records are stored in pages.

Chapter roadmap

#	Chapter	Question it answers	Main directories
01	Files, Tablespaces & Space Management	Where do bytes live on disk? How are pages allocated?	fil/, fsp/, os/
02	The 16KB Page & Record Format	What exactly is inside a page? How is a row encoded?	page/, rem/
03	The Buffer Pool	How does InnoDB cache pages and decide what to evict/flush?	buf/
04	Mini-Transactions & Latching	How is a single page change made atomic and logged?	mtr/, sync/
05	Redo Log & Crash Recovery	How does InnoDB survive a crash without losing commits?	log/
06	The B+Tree	How are rows indexed, found, inserted, and pages split?	btr/, page/
07	Transactions, Undo & MVCC	How do commit, rollback, and consistent reads work?	trx/, read/
08	The Lock Manager	How are conflicts and phantoms prevented? How are deadlocks found?	lock/
09	Row Operations	What happens end-to-end on INSERT/SELECT/UPDATE/DELETE?	row/
10	The Data Dictionary	Where does InnoDB keep its own schema?	dict/, pars/, que/
11	Startup, Shutdown & the Embedded API	What happens between <code>ib_init()</code> and a usable database?	api/, srv/
12	Background Threads & the Master Thread	Who does the work your transaction didn't wait for?	srv/, ibuf/, trx/

How to study with this series

1. **Read a chapter, then read the code it cites.** Each chapter references functions as `file.c` (in the module directory) — e.g. `buf/buf0buf.c` — and structs in `include/*.h`. Remember: many functions are defined in `include/*.ic` inline files, not the `.c` file.
2. **Run the tests and trace them.** The `tests/` directory has small programs (`ib_test1.c`, `ib_cursor.c`) that exercise everything. Trace API calls with `ltrace -e 'ib_*' .libs/ib_test1` or use `tests/debug_gdb.sh` to break inside the engine.
3. **Look at real bytes.** After running a test, hex-dump `ibdata1` (`xxd tests/ibdata1 | less`) and find the structures from chapters 1-2 at their documented offsets.
4. **Follow one operation all the way down.** A good capstone: pick `ib_cursor_insert_row()` and walk it through `api` → `row` → `btr` → `page` → `mtr` → `log` using chapters 9, 6, 4, 5.

Conventions used in this series

- **File references** are relative to the repository root: `btr/btr0cur.c`, `include/trx0trx.h`.
 - **Naming:** files are `module0submodule.c`; functions are `module_submodule_action()`. So `buf_LRU_get_free_block()` lives in `buf/buf0lru.c`.
 - **Page size** is fixed: `UNIV_PAGE_SIZE = 16384` bytes (`include/univ.i`).
 - Diagrams are Mermaid (rendered by GitHub/VS Code) or ASCII byte-layout tables.
-

Next: [Chapter 1 — Files, Tablespaces & Space Management](#)

Chapter 1 — Files, Tablespaces & Space Management

Layer 1 of 5 — Storage. Where bytes live on disk, and how InnoDB carves files into pages, extents, and segments. Source: `fil/fil0fil.c`, `fsp/fsp0fsp.c`, `include/fil0fil.h`, `include/fsp0types.h`, `os/`

1.1 The unit of everything: the page

InnoDB never reads or writes individual bytes from disk. The atomic unit of I/O and storage is the **page**: `UNIV_PAGE_SIZE` = 2^{14} = **16384 bytes** (`include/univ.i:242-244`).

Every page in the system has a global address:

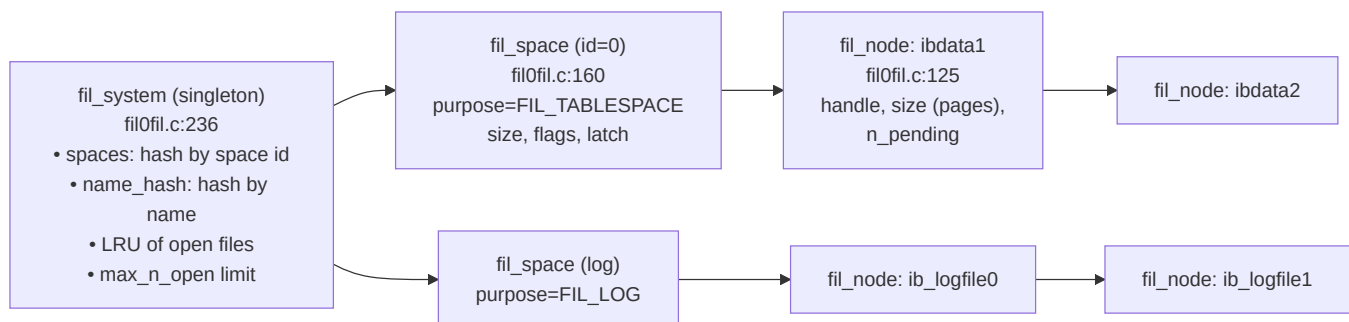
```
(space id, page number) → byte offset = page_number × 16384 within the space's file(s)
```

- **space id** — which tablespace (0 = the system tablespace, `ibdata1`)
- **page number** — 4-byte page index within that space; `FIL_NULL` (0xFFFFFFFF) means "no page" (`include/fil0fil.h:47`)

Some on-disk structures point not just to a page but *into* one. That is a **file address** (`fil_addr_t`, `include/fil0fil.h:65-68`): a page number plus a 2-byte byte-offset within the page — 6 bytes on disk. File addresses are what on-disk linked lists (`fut/fut0lst.c`) are made of.

1.2 The FIL layer: mapping spaces to real files

The `fil/` module ("file space abstraction") maintains the mapping from logical (`space`, `page_no`) addresses to actual OS files. Three structs, all in `fil/fil0fil.c`:



- `fil_system_struct` (`fil/fil0fil.c:236-283`) — the singleton cache of all tablespaces. It hashes spaces by id and by name, and keeps an LRU of open file handles so InnoDB can run with more tablespaces than the OS allows open file descriptors (`max_n_open`).
- `fil_space_struct` (`fil/fil0fil.c:160-224`) — one tablespace or one log space (`purpose = FIL_TABLESPACE` or `FIL_LOG`). Holds total `size` in pages and a chain of files.
- `fil_node_struct` (`fil/fil0fil.c:125-154`) — one physical file. A space with several data files (e.g. `ibdata1;ibdata2`) has several nodes chained in order; page numbers run consecutively across the chain.

New tablespaces register through `fil_space_create(name, id, flags, purpose)` (`fil/fil0fil.c:1157`).

How an I/O actually happens: `fil_io()`

All page I/O funnels through one function, `fil_io()` (`fil/fil0fil.c:4326`):

```
caller (buffer pool, log)
└─ fil_io(type, sync, space_id, zip_size, block_offset, byte_offset, len, buf, msg)
    1. space = fil_space_get_by_id(space_id)           ← hash lookup   (:4414)
    2. walk space->chain, subtracting node->size,
       until the node containing block_offset is found (:4433)
    3. file offset = block_offset * 16384 + byte_offset (:4480)
    4. os_aio(...) → async I/O, or immediate if sync   (:4521)
```

The `os/` layer (`os/os0file.c`) hides platform differences and implements **simulated asynchronous I/O**: dedicated I/O handler threads sit in `fil_aio_wait()` waiting for completions, so the rest of the engine can issue reads/writes without blocking. This is the 1990s-era answer to portable async I/O, and it still shapes InnoDB's thread architecture (see Chapter 12).

1.3 The FSP layer: allocating pages inside a space

The `fil/` layer knows how to *reach* a page; the `fsp/` layer ("file space management") decides which pages are *used for what*. It manages three granularities:

```
tablespace
└─ extent   = 64 consecutive pages = 1 MB      (FSP_EXTENT_SIZE, include/fsp0types.h:45)
    └─ page  = 16 KB
segment (fseg) = a logical file inside the space, owning fragment pages + whole extents
```

The space header (page 0)

Page 0 of every tablespace is type `FIL_PAGE_TYPE_FSP_HDR`. After the standard 38-byte FIL header (Chapter 2) comes the **FSP header** at offset 38 (`fsp/fsp0fsp.c:70-110`):

offset (in header)	field	meaning
0	FSP_SPACE_ID	this space's id
8	FSP_SIZE	current size in pages
12	FSP_FREE_LIMIT	first page never yet initialized
16	FSP_SPACE_FLAGS	format/compression flags
20	FSP_FRAG_N_USED	used pages in FREE_FRAG extents
24	FSP_FREE	list base: completely free extents
40	FSP_FREE_FRAG	list base: extents with some pages used individually
56	FSP_FULL_FRAG	list base: fragment extents fully used
72	FSP_SEG_ID	next segment id to assign (8 bytes)
80	FSP_SEG_INODES_FULL	list base: full segment-inode pages
96	FSP_SEG_INODES_FREE	list base: inode pages with free slots

The "list base" entries are on-disk doubly-linked lists (`ftut/ftut0lst.c`): a 16-byte base node (`FLST_BASE_NODE_SIZE`, `count + first + last fil_addr_t`) whose 12-byte list nodes (`FLST_NODE_SIZE`, `prev + next`) live inside the linked structures themselves. **This is a key pattern: InnoDB builds linked lists *across disk pages* out of file addresses.**

Extent descriptors (XDES)

Right after the FSP header, at byte offset 150 of page 0, sits an array of **extent descriptors** (`fsp/fsp0fsp.c:201-234`), 40 bytes each — one per extent:

XDES entry (40 bytes)

XDES_ID	list node	state	bitmap: 2 bits × 64 pages
owning seg	(12 bytes)		(FREE bit + unused CLEAN bit)
(8 bytes)			(16 bytes)

states: XDES_FREE(1) XDES_FREE_FRAG(2) XDES_FULL_FRAG(3) XDES_FSEG(4)

One page of descriptors covers 16384 extents = 256MB, so the descriptor page repeats: every 16384th page (page 0, 16384, 32768, ...) is an XDES page (type `FIL_PAGE_TYPE_XDES`), and the page after it is an insert-buffer bitmap page (`FSP_IBUF_BITMAP_OFFSET` = 1, Chapter 12). That's why those page numbers are "stolen" in every tablespace.

Fixed low pages of the system tablespace

The system tablespace reserves its first pages for engine metadata (`include/fsp0types.h:88-103`):

page	constant	contents	chapter
0	—	FSP header + XDES array	this one
1	FSP_IBUF_BITMAP_OFFSET	insert buffer bitmap	12
2	FSP_FIRST_INODE_PAGE_NO	first segment-inode page	this one
3	FSP_IBUF_HEADER_PAGE_NO	insert buffer header	12
4	FSP_IBUF_TREE_ROOT_PAGE_NO	insert buffer B+tree root	12
5	FSP_TRX_SYS_PAGE_NO	transaction system header (+ doublewrite info)	7
6	FSP_FIRST_RSEG_PAGE_NO	first rollback segment header	7
7	FSP_DICT_HDR_PAGE_NO	data dictionary header	10

You can see these with `xxd tests/ibdata1 | less` after running a test — e.g. page 5 starts at byte offset $5 \times 16384 = 0x14000$.

Segments (fseg): logical files inside the space

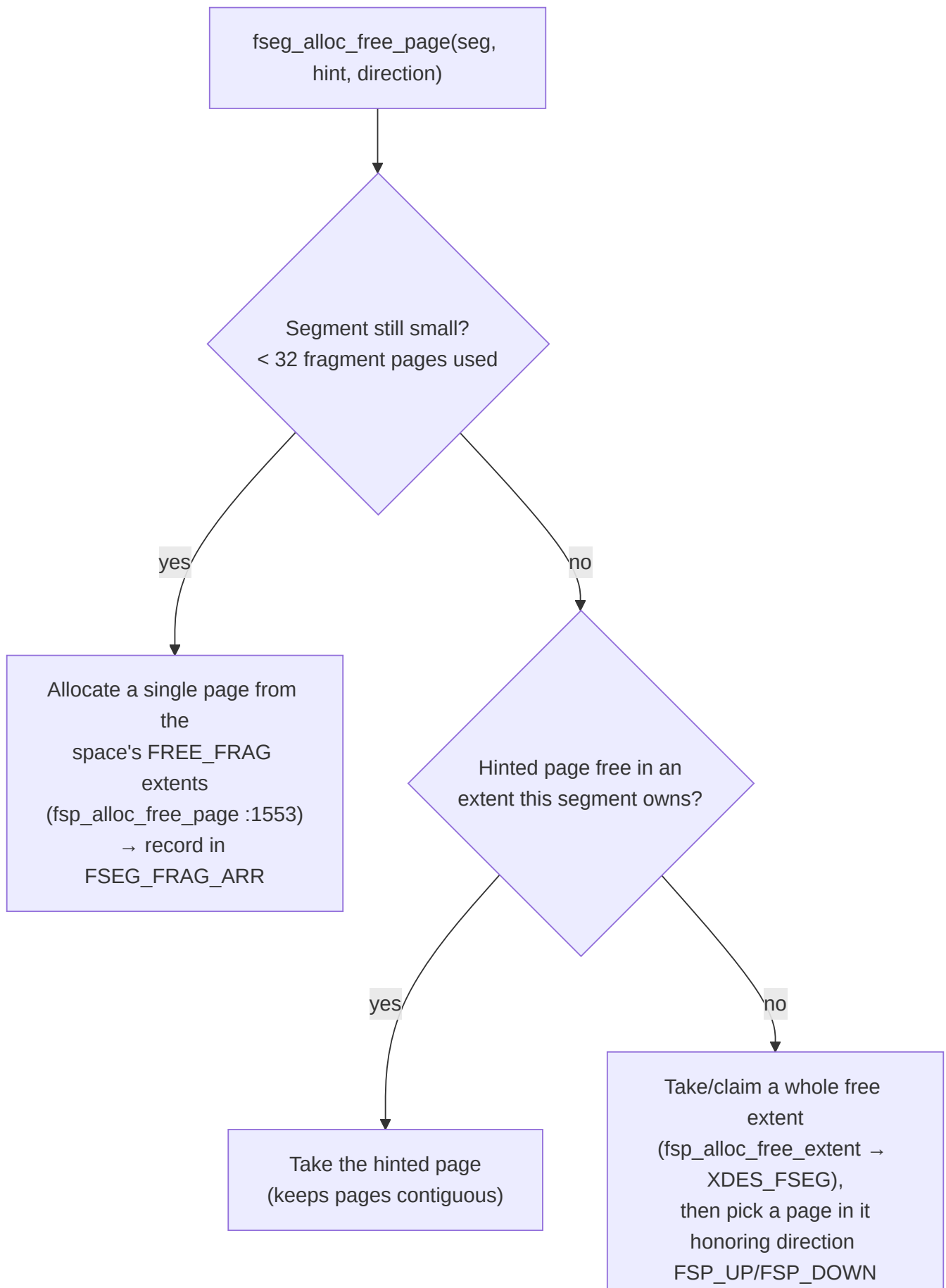
A **segment** is what higher layers actually allocate from. Every B+tree index uses *two* segments — one for leaf pages, one for internal pages (Chapter 6); undo logs use segments too (Chapter 7). A segment is described by a 192-byte **inode** stored on an inode page (`fsp/fsp0fsp.c:116-192`, 85 inodes fit per page):

field	meaning
FSEG_ID	segment id (0 = free slot)
FSEG_FREE	list base: extents owned, fully free
FSEG_NOT_FULL	list base: extents owned, partially used
FSEG_FULL	list base: extents owned, full
FSEG_NOT_FULL_N_USED	used-page count in NOT_FULL extents
FSEG_FRAG_ARR	array of 32 individual "fragment" page numbers

Users of a segment hold a 10-byte **segment header** (`FSEG_HEADER_SIZE`, `include/fsp0types.h:57-62`) = (space, inode page, inode offset) — e.g. stored in a B+tree root page — which is simply a pointer to the inode.

The allocation strategy: fragments first, then whole extents

`fseg_alloc_free_page()` (`fsp/fsp0fsp.c:2898` → `fseg_alloc_free_page_low()` :2579) embodies a deliberate small-to-large growth policy:



The idea: a tiny table shouldn't consume 1MB extents, so its first 32 pages come one at a time from shared fragment extents. Once a segment proves it is growing (fill factor check against `FSEG_FILLFACTOR = 8`, `FSEG_FRAG_LIMIT = 32`), it graduates to owning whole 1MB extents — which is what makes large scans sequential on disk. The `hint` and `direction` parameters let the B+tree keep sibling pages physically close.

To avoid deadlocking on a full disk, multi-page operations first call `fsp_reserve_free_extents()` (`fsp/fsp0fsp.c`, alloc types `FSP_NORMAL`/`FSP_UNDO`/`FSP_CLEANING`) so that cleanup operations always have room to proceed.

1.4 What to remember

1. Everything is a **16KB page** addressed by `(space id, page number)`; `fil_io()` is the single funnel from page addresses to file offsets to (simulated) async I/O.
2. Space management is **bitmap-based**: XDES entries with 1 free-bit per page, grouped into 64-page/1MB extents.
3. **Segments** give each index/undo-log its own logical file, growing from shared fragment pages to private extents — the mechanism behind InnoDB's good locality.
4. On-disk **linked lists of file addresses** (`fut0lst`) stitch all of this together; you will meet them again in undo logs and the insert buffer.

Try it: run `tests/.libs/ib_test1`, then `xxd tests/ibdata1 | awk 'NR<=16'` — bytes 24–25 of page 0 are `0x0008` (`FIL_PAGE_TYPE_FSP_HDR`), and at offset `38+8` you'll find `FSP_SIZE`.

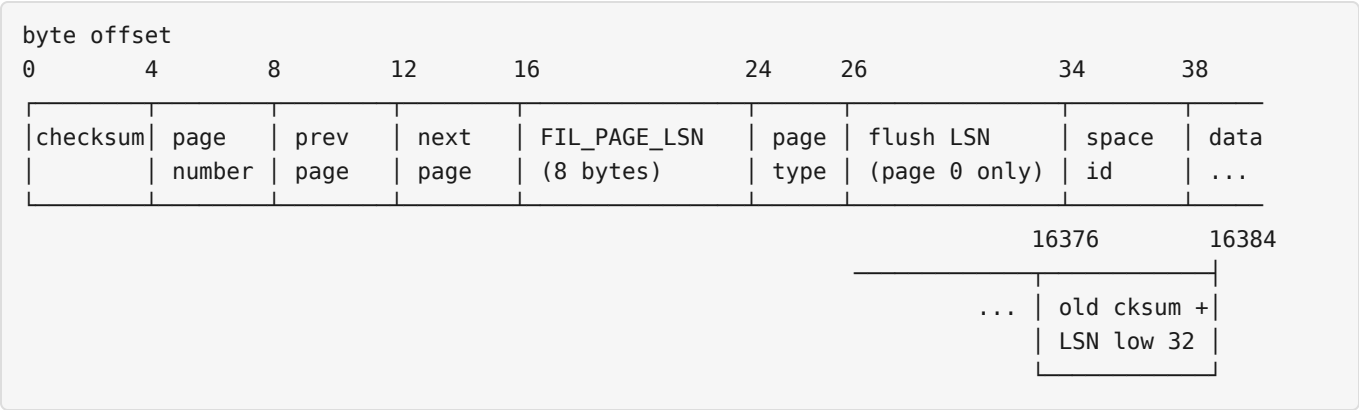
Previous: [Chapter 0 — Overview](#) · **Next:** [Chapter 2 — The 16KB Page & Record Format](#)

Chapter 2 — Inside the 16KB Page & the Record Format

Layer 1 of 5 — Storage. The anatomy of an index page and the byte-level encoding of a row. Source: `include/fil0fil.h`, `include/page0page.h`, `page/page0page.c`, `rem/rem0rec.c`, `include/rem0rec.h`

2.1 Every page starts and ends the same way

Regardless of type, every 16384-byte page carries a **38-byte FIL header** and an **8-byte FIL trailer** (`include/fil0fil.h:74-127`):



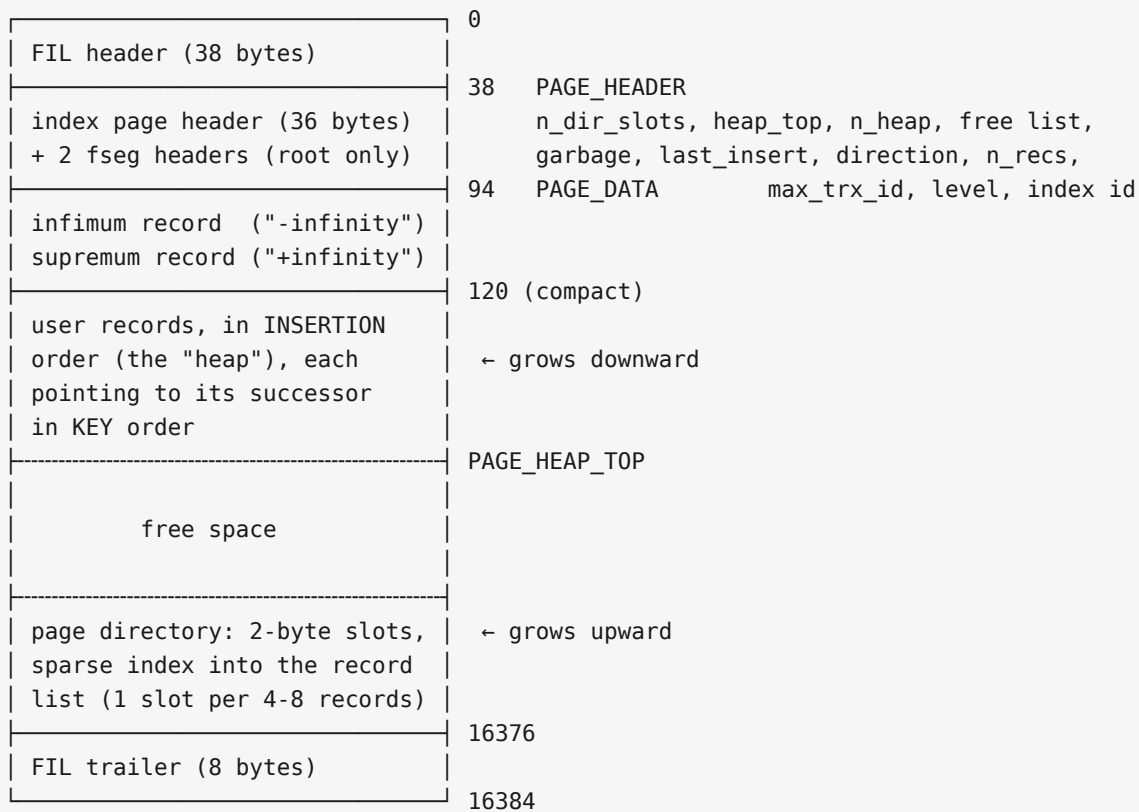
Three fields matter enormously for later chapters:

- **FIL_PAGE_LSN (offset 16)** — the log sequence number of the *newest change* to this page. Crash recovery compares this against redo records to know what is already applied (Chapter 5).
- **FIL_PAGE_PREV / FIL_PAGE_NEXT (offsets 8/12)** — sibling pointers. On B+tree leaf pages these form the doubly-linked list that makes range scans possible (Chapter 6).
- **FIL_PAGE_TYPE (offset 24)** — e.g. `FIL_PAGE_INDEX` (17855) for B+tree pages, `FIL_PAGE_UNDO_LOG` (2), `FIL_PAGE_TYPE_FSP_HDR` (8) (`include/fil0fil.h:130-144`).

The trailer repeats the low 32 bits of the LSN. If a write is torn mid-page, header LSN $\neq$ trailer LSN — that's how `buf_page_is_corrupted()` (`buf/buf0buf.c:369`) detects partial writes. The checksum (`buf_calc_page_new_checksum`, `buf/buf0buf.c:317`) covers everything except itself and the trailer, and is stamped at flush time (`buf_flush_init_for_writing`, `buf/buf0flu.c:704-725`).

2.2 The index page: a heap of records plus a sorted directory

A B+tree page (`FIL_PAGE_INDEX`) has this internal structure:

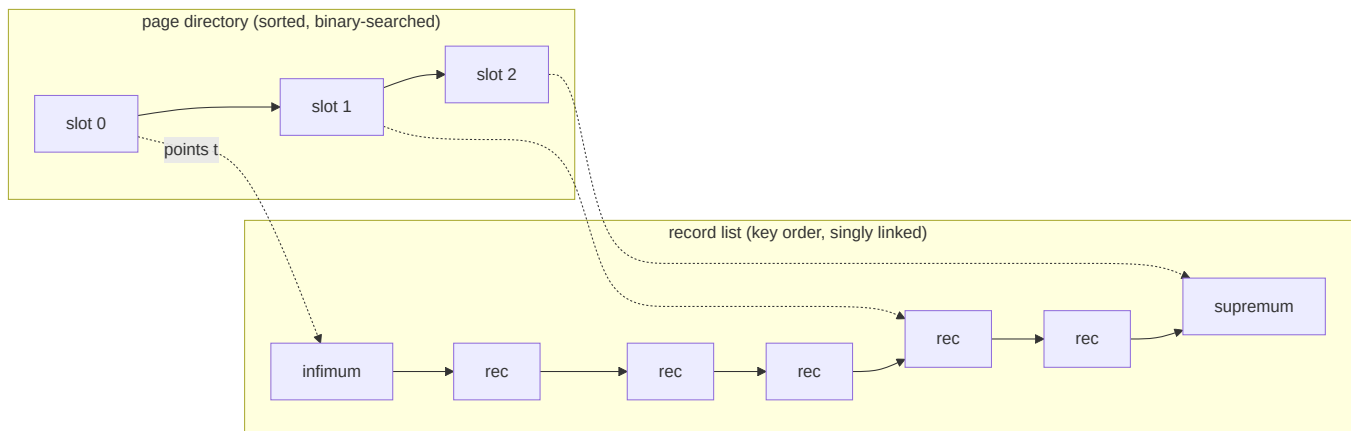


Key fields of the 36-byte page header (offsets relative to `PAGE_HEADER = 38`,
`include/page0page.h:55-98`):

offset	field	meaning
0	<code>PAGE_N_DIR_SLOTS</code>	number of directory slots
2	<code>PAGE_HEAP_TOP</code>	first byte after the record heap
4	<code>PAGE_N_HEAP</code>	records in heap; bit 15 set = compact format
6	<code>PAGE_FREE</code>	head of the free (deleted) record list
8	<code>PAGE_GARBAGE</code>	bytes in deleted records (for reorganize decision)
10	<code>PAGE_LAST_INSERT</code>	position of last insert (split heuristic, Ch. 6)
12	<code>PAGE_DIRECTION</code>	recent insert direction (<code>PAGE_LEFT</code> / <code>PAGE_RIGHT</code> ...)
16	<code>PAGE_N_RECS</code>	user record count
18	<code>PAGE_MAX_TRX_ID</code>	max trx that modified this page (secondary indexes; Ch. 7)
26	<code>PAGE_LEVEL</code>	tree level: 0 = leaf
28	<code>PAGE_INDEX_ID</code>	which index this page belongs to
36	<code>PAGE_BTR_SEG_LEAF/TOP</code>	fseg headers (root page only; Ch. 1, Ch. 6)

The three clever ideas in this layout

1. **Records live where they were inserted; order lives in pointers.** Records are appended to a heap (growing down from `PAGE_DATA`), and each record's header contains a *next-record pointer* forming a **singly-linked list in key order**: `infimum` → `smallest` ... `largest` → `supremum`. Inserting never shifts other records — it just splices the list. Deleted record space goes on the `PAGE_FREE` list for reuse.
2. **Infimum and supremum are real records.** Two system records, fixed at bytes 99/112 (compact; `include/page0page.h:112-118`), act as $-\infty$ and $+\infty$ sentinels. Every algorithm can assume the list has a first and last element, eliminating edge cases; supremum also serves as the lock target for "the gap after the last record" (Chapter 8).
3. **The page directory makes search $O(\log n)$.** A linked list alone would force linear scans. The directory — 2-byte slots growing up from the trailer — points at every 4th-to-8th record (`PAGE_DIR_SLOT_MIN/MAX_N_OWNED = 4/8`, `include/page0page.h:142-161`). Searching a page (`page_cur_search_with_match`, `page/page0cur.c`) binary-searches the directory, then walks at most ~ 8 list nodes. Each record's `n_owned` field says how many records its directory slot covers.



2.3 The record format: how a row is encoded

InnoDB has two record formats, and this codebase contains the transition between them:

- **Redundant (old-style)** — original format, 6 extra header bytes (`REC_N_OLD_EXTRA_BYTES`, `include/rem0rec.h:46`), stores an offset for every field.
- **Compact (new-style)** — 5 extra header bytes (`REC_N_NEW_EXTRA_BYTES`, `include/rem0rec.h:49`), stores lengths only for variable-length fields and a NULL bitmap. A page is compact if bit 15 of `PAGE_N_HEAP` is set.

A compact record looks like:

└ record pointer ("origin") points HERE



var-len lengths (reversed)	NULL bits	5-byte header (read at negative offsets)	field 1 data	field 2 data	...
----------------------------------	--------------	---	-----------------	-----------------	-----

5-byte header contains:

- info bits: REC_INFO_DELETED_FLAG (0x20 = delete-marked)
- REC_INFO_MIN_REC_FLAG (0x10 = min rec on level)
- n_owned (records owned by my directory slot)
- heap_no (position in page heap; 13 bits)
- status (3 bits: ORDINARY / NODE_PTR / INFIMUM / SUPRENUM)
- next-record pointer (2 bytes, relative offset)

(Bit layout: `include/rem0rec.h:39-64`; accessors like `rec_get_heap_no_new`, `rec_get_next_offs` at `include/rem0rec.h:96-306` — note they read *before* the origin.)

Points worth internalizing:

- **The delete flag, not deletion.** `REC_INFO_DELETED_FLAG` marks a record deleted without removing it. MVCC readers may still need it, and its key must remain to prevent phantoms. Actual removal happens later, by purge (Chapter 7).
- **The record pointer points at the data, header behind it.** All header accessors use negative offsets. This lets field access proceed without knowing header size.
- **status distinguishes leaf records from node pointers.** Internal (non-leaf) B+tree pages hold `REC_STATUS_NODE_PTR` records: a key prefix + 4-byte child page number (`REC_NODE_PTR_SIZE`, `include/rem0rec.h:52-55`). Same page machinery, different payload — that's how one page format serves every level of the tree.

Hidden system columns

Every clustered-index (primary key) leaf record carries columns your schema never declared:

column	size	purpose
<code>DB_ROW_ID</code>	6 bytes	synthetic PK, only when the table has no user PK
<code>DB_TRX_ID</code>	6 bytes	id of the transaction that last modified this row
<code>DB_ROLL_PTR</code>	7 bytes	pointer to the undo record holding the <i>previous version</i>

`DB_TRX_ID` + `DB_ROLL_PTR` are the anchors of MVCC: they turn each row into the head of a version chain threaded through the undo log (Chapter 7).

2.4 What to remember

1. Pages are self-describing (type, LSN, checksums) and self-verifying (torn-write detection via header/trailer LSN match).

2. Within an index page: **heap for placement, linked list for order, directory for speed** — an insert costs a splice, a search costs a binary search.
3. Records carry their bookkeeping (delete flag, heap number, next pointer) in a header *behind* the record origin; clustered rows additionally carry `DB_TRX_ID/DB_ROLL_PTR`, the hooks for the entire MVCC machinery.
4. This codebase shows both the redundant and compact formats, plus their parallel redo record types (`MLOG_REC_INSERT` vs `MLOG_COMP_REC_INSERT`, Chapter 4) — a lesson in how a storage engine evolves a format without breaking old data.

Try it: run a test, then dump a data page: `xxd -s $((N*16384)) -l 512 tests/ibdata1` for increasing N until byte 24-25 reads `0x45bf` (17855 = `FIL_PAGE_INDEX`). Bytes 38+16..17 are `PAGE_N_RECS`.

Previous: [Chapter 1 — Files & Tablespaces](#) · **Next:** [Chapter 3 — The Buffer Pool](#)

Chapter 3 — The Buffer Pool

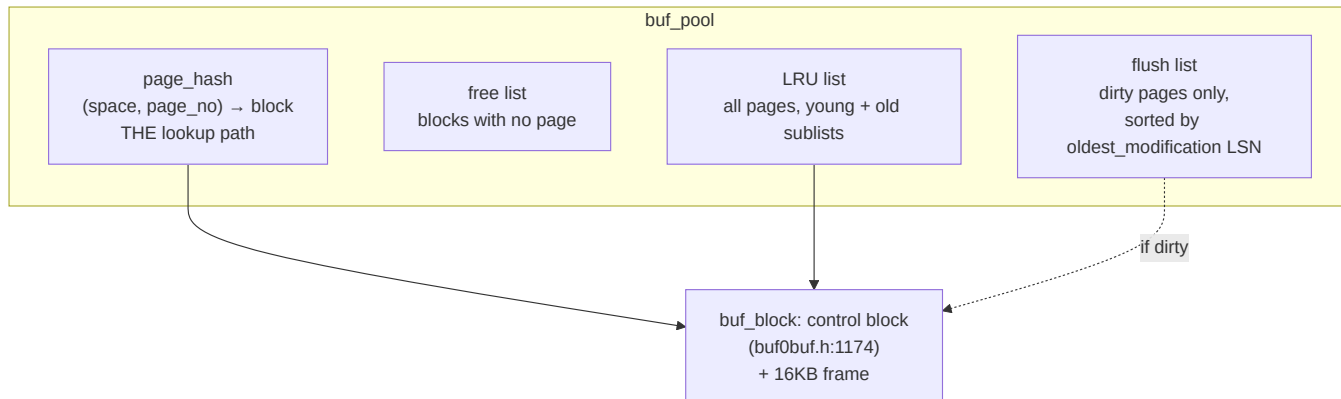
Layer 2 of 5 — Caching. How InnoDB keeps hot pages in memory, decides what to evict, and writes dirty pages back safely. Source: `buf/buf0buf.c`, `buf/buf0lru.c`, `buf/buf0flu.c`, `buf/buf0rea.c`, `include/buf0buf.h`

3.1 Why a buffer pool

Every page access in InnoDB — B+tree descent, undo lookup, dictionary read — goes through the buffer pool. Nothing above the `buf/` layer ever calls `fil_io()` directly. This single choke-point gives InnoDB three things at once: caching, **write-ahead-logging enforcement** (a dirty page cannot reach disk before its redo log does — Chapter 5), and **torn-write protection** (the doublewrite buffer, below).

3.2 The data structures

The singleton `buf_pool` (`include/buf0buf.h:1337-1446`) owns a set of memory chunks divided into 16KB frames, each with a control block, and four bookkeeping structures:



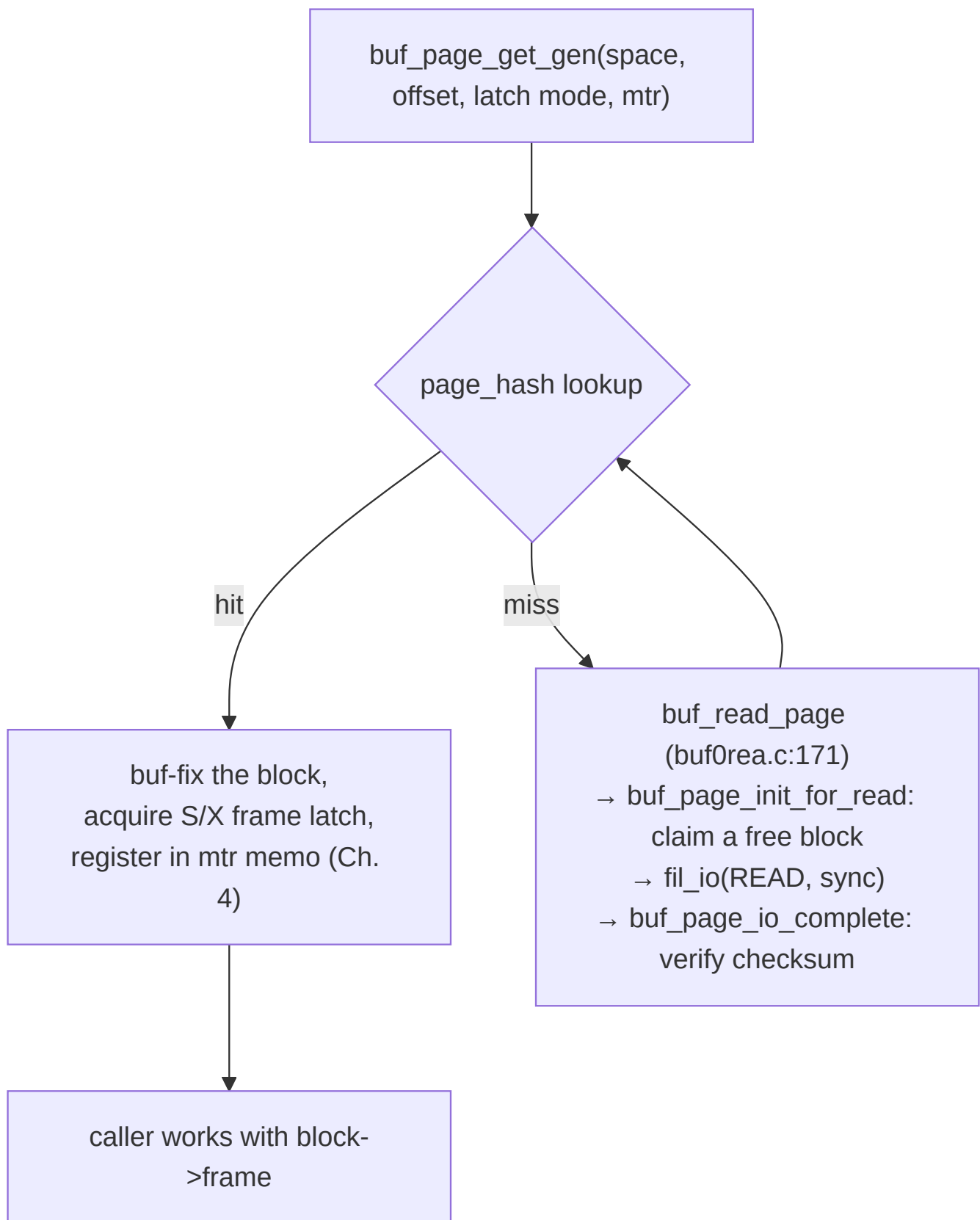
Each page's control block (`buf_page_struct`, `include/buf0buf.h:1040-1169`) carries the fields that drive everything else:

field	meaning
space, offset	page identity — the hash key
state	BUF_BLOCK_NOT_USED (free), BUF_BLOCK_FILE_PAGE (caching a page), ...
io_fix, buf_fix_count	page pinned for I/O / by users — cannot be evicted
oldest_modification	LSN of the <i>first</i> redo record since the page was last clean — position in flush list; 0 = clean
newest_modification	LSN of the latest change — the WAL barrier for flushing
old, access_time	LRU old-sublist membership and first-access time

The full `buf_block_struct` adds the frame pointer, a per-block mutex, the **frame rw-lock** (this is the "page latch" every other chapter talks about), and `modify_clock` — a counter bumped on every reorganization, which lets cursors optimistically revalidate a remembered position without a fresh tree descent (Chapter 6).

3.3 Fetching a page: `buf_page_get_gen()`

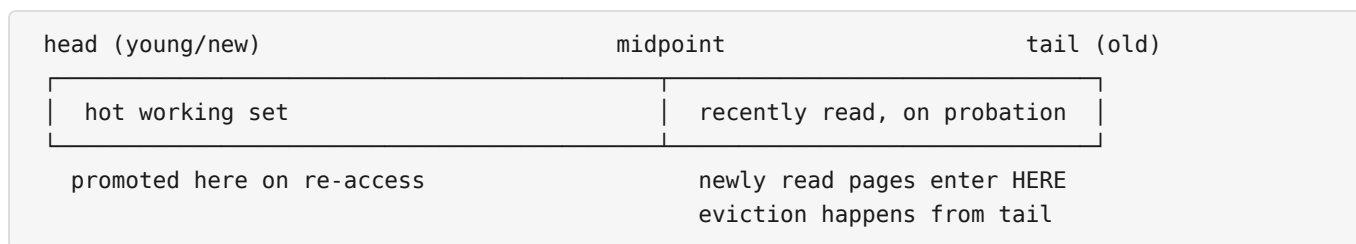
The workhorse (`buf/buf0buf.c:2134`) — the function behind every `buf_page_get()` you'll see in `btr/row/trx` code:



If the free list is empty, `buf_LRU_get_free_block()` (`buf/buf0lr.c`) evicts from the LRU tail — writing the victim out first if dirty (an "LRU flush").

The midpoint LRU

A naive LRU has a famous failure mode: one large table scan evicts the entire working set. InnoDB's answer is a **two-part LRU** (`buf/buf0lru.c`):



- Newly read pages enter at the **midpoint** (head of the old sublist), not the head.
- Only a *subsequent* access promotes a page to the young half — and only if it has survived `buf_LRU_old_threshold_ms` (`buf/buf0lru.c:124`), so a scan that touches each page once (or twice within milliseconds) never pollutes the young half.
- The old sublist's share is `buf_LRU_old_ratio/1024` of the list (`buf/buf0lru.c:121`, bounds $\approx 5\%$ – 100% , `include/buf0lru.h:245-255`); the split only exists once the LRU exceeds `BUF_LRU_OLD_MIN_LEN = 512` pages (`include/buf0lru.h:72`).

The API exposes these as `lru_old_blocks_pct` and `lru_block_access_recency` (`api/api0cfg.c:880-890`) — this very engine is where MySQL's `innodb_old_blocks_pct/innodb_old_blocks_time` come from.

Read-ahead

`buf_read_ahead_linear()` (`buf/buf0rea.c:235`): when the accessed page is the border page of a 64-page area and every page of that area has been accessed, InnoDB predicts a sequential scan and prefetches the next area asynchronously. (This version has only linear read-ahead; the random read-ahead of later versions is absent — a nice example of a feature that was removed and re-added over InnoDB's history.)

3.4 Writing pages back: the flush list

A page becomes dirty inside a mini-transaction commit (Chapter 4), which sets `oldest/newest_modification` and inserts the block into the **flush list, ordered by `oldest_modification`**. That ordering is the whole point: the minimum `oldest_modification` across the flush list tells the checkpoint algorithm (Chapter 5) exactly how much redo log is still "needed", so flushing from the flush-list tail *advances the checkpoint*.

Flushing comes in flavors (`enum buf_flush`, `include/buf0types.h:44-49`):

- **BUF_FLUSH_LIST** — flush pages with the oldest modification LSNs; driven by the master thread and by checkpoint pressure (Chapters 5, 12).
- **BUF_FLUSH_LRU** — flush/evict from the LRU tail to keep free blocks available.
- **BUF_FLUSH_SINGLE_PAGE** — emergency single-page flush.

Each write goes through `buf_flush_write_block_low()` (`buf/buf0flu.c:733`), which enforces two invariants in order:

1. **WAL first:** `log_write_up_to(bpage->newest_modification, ..., TRUE)` (`buf/buf0flu.c:771`) — the redo log describing the page's latest change is flushed and fsynced *before* the page itself. This one line is the write-ahead-logging rule.
2. **Stamp and checksum:** `buf_flush_init_for_writing()` (`buf/buf0flu.c:650`) writes `FIL_PAGE_LSN`, the trailer LSN, and both checksums into the frame.

The doublewrite buffer

A 16KB page write is not atomic on most storage: a crash mid-write leaves a "torn" page that checksums cannot repair (redo records describe *changes*, not full pages, so a half-written page is unrecoverable). InnoDB's fix — invented in this era and still in use today — is the **doublewrite buffer**: 2×64 pages reserved in the system tablespace, described from the `TRX_SYS` page (`trx_doublewrite`, `trx/trx0sys.c:161`, layout `include/trx0sys.h:445-523`).

flush a batch of dirty pages:

1. memcpy pages into the doublewrite staging buffer
2. write staging buffer sequentially to the doublewrite area + fsync
3. write each page to its real location + fsync

recovery: for each page in the doublewrite area,
if the real page is torn (bad checksum) → restore it from the copy

Either copy of the page survives any single crash intact, at the cost of writing everything twice (cheap, because step 2 is one sequential write).

3.5 What to remember

1. The buffer pool is the single gateway between memory and disk; its per-page metadata (`oldest/newest_modification`, frame rw-lock, `modify_clock`) is infrastructure that *other* subsystems — logging, checkpointing, B+tree cursors — depend on.
2. The **midpoint LRU** protects the working set from scans; you are looking at the original implementation of a design MySQL DBAs still tune today.
3. The **flush list is LSN-ordered**, which couples page writeback directly to checkpoint advancement — the buffer pool and the redo log are two halves of one mechanism.
4. The **doublewrite buffer** solves torn pages; the WAL call in `buf_flush_write_block_low()` solves lost changes. Together they make "crash anywhere" safe.

Try it: `grep -n "log_write_up_to" buf/buf0flu.c` — one call, and it's the entire WAL guarantee.

Previous: [Chapter 2 — Page & Record Format](#) · **Next:** [Chapter 4 — Mini-Transactions & Latching](#)

Chapter 4 — Mini-Transactions & Latching

Layer 3 of 5 — Durability. The mechanism that makes every physical page change atomic *and* logged — the hinge between the buffer pool and the redo log. Source: `mtr/mtr0mtr.c`, `mtr/mtr0log.c`, `include/mtr0mtr.h`, `include/sync0sync.h`, `sync/sync0rw.c`

4.1 The problem mini-transactions solve

A logical operation like "insert a row" may touch several pages: the leaf page, maybe a split sibling, a node pointer page, the space bitmap. Two dangers:

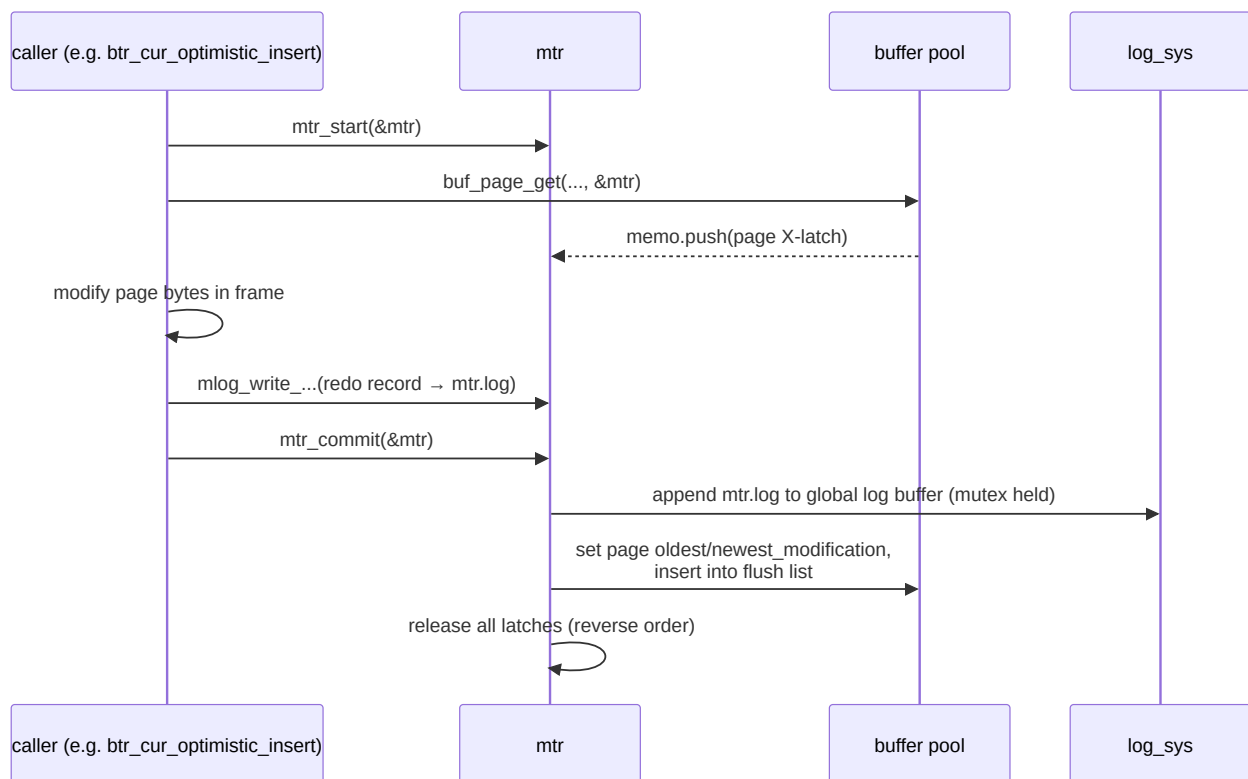
1. **Concurrency** — another thread must never see a page *between* two related changes (e.g. a leaf split half-done).
2. **Durability** — if we crash mid-change, recovery must either redo the whole group of page changes or none of it.

The **mini-transaction** (`mtr_t`) solves both with one object. Despite the name it has nothing to do with user transactions (no rollback, no locks); it is a short-lived *critical section + redo-log batch*. Every function in `btr/`, `fsp/`, `trx/`, `ibuf/` that touches a page takes an `mtr_t*` parameter — now you know why.

4.2 Anatomy and lifecycle

`mtr_t` (`include/mtr0mtr.h:376-397`) is two growable arrays and a bit of state:

- `memo` — a stack of every latch acquired (page S/X-latches, index tree locks) and every page modified.
- `log` — the redo records describing the changes, buffered locally.
- `n_log_recs`, `modifications`, `start_lsn/end_lsn`.



The three rules a caller must obey:

1. Latch every page you touch through the `mtr` (`buf_page_get` registers it in the memo).
2. Never modify page bytes directly — use the `mlog_*` writers, which change the page *and* append the matching redo record.
3. Keep `mtrs` short. Latches are held until `mtr_commit()`; a long `mtr` is a concurrency bottleneck.

What commit does — and why the order matters

`mtr_commit()` (`mtr/mtr0mtr.c:176-218`):

1. If more than one log record was written, append `MLOG_MULTI_REC_END`; if exactly one, OR `MLOG_SINGLE_REC_FLAG` into its type byte (`mtr/mtr0mtr.c:129-131`). **This is the atomicity marker:** recovery only applies a group of records if it sees the complete group ending in `MLOG_MULTI_REC_END` (Chapter 5). Crash mid-`mtr` → incomplete group → recovery ignores it → the multi-page change never happened. That's how a page split is all-or-nothing.
2. Acquire the log mutex, copy `mtr.log` into the global log buffer; this assigns the `mtr` its `start_lsn/end_lsn` (`mtr_log_reserve_and_write`, `mtr/mtr0mtr.c:110-171`).
3. **Still under the log mutex**, mark all modified pages dirty (setting their modification LSNs, inserting into the flush list) and release the page latches (`mtr/mtr0mtr.c:200-206`). Doing this before releasing the log mutex guarantees that whenever the log mutex is free, every buffer page's modification info is current — the checkpoint calculation (Chapter 5) depends on exactly this invariant.

4.3 Redo records: physiological logging

The records in `mtr.log` are typed by one byte (`MLOG_*`, `include/mtr0mtr.h:57-183`). A record's shape is:

```
type (1) | space id (1-5, compressed) | page number (1-5, compressed) | body...
```

The types span a spectrum from physical to logical:

type	value	meaning	body
<code>MLOG_1BYTE/2BYTES/4BYTES/8BYTES</code>	1,2,4,8	write N bytes at offset	offset + value
<code>MLOG_WRITE_STRING</code>	30	write byte string	offset + len + bytes
<code>MLOG_REC_INSERT</code> / <code>MLOG_COMP_REC_INSERT</code>	9 / 38	insert record into page	record image
<code>MLOG_REC_DELETE</code> , <code>MLOG_REC_UPDATE_IN_PLACE...</code>	14, 13	record ops	
<code>MLOG_PAGE_CREATE</code> / <code>MLOG_COMP_PAGE_CREATE</code>	19 / 37	(re)initialize an index page	—
<code>MLOG_UNDO_INSERT</code> , <code>MLOG_UNDO_HDR_CREATE...</code>	20, 25	undo-log page ops	
<code>MLOG_INIT_FILE_PAGE</code>	29	page allocated: prior contents void	—
<code>MLOG_FILE_CREATE/RENAME/DELETE</code>	33-35	file operations	name
<code>MLOG_MULTI_REC_END</code>	31	end of an atomic group	—

This is called **physiological logging**: physical *to a page* (space+page_no), logical *within the page* ("insert this record"), rather than logging full page images (too big) or purely logical SQL (not idempotent, needs a consistent structure to apply to). A `MLOG_REC_INSERT` replayed against the page re-runs the actual insert code (`page_cur_insert_rec_low`) during recovery — the same routine used at runtime, just fed from the log. Look at any `page0cur.c` or `btr0cur.c` operation and you'll find the pattern: `foo()` does the work and writes the record; `foo_parse_log()/recv_parse_or_apply_log_rec_body` replays it.

Note the parallel `MLOG_COMP_*` series (36-46): when the compact record format was added (Chapter 2), every record-level redo type needed a sibling. Log formats are forever.

4.4 Latching: rw-locks and the ordering discipline

The latches in the mtr memo are **rw-locks** (`sync/sync0rw.c`, `include/sync0rw.h`) — this version supports S (shared) and X (exclusive, recursive for the same thread) modes, implemented over one atomic `lock_word` (readers decrement by 1; a writer subtracts `X_LOCK_DECR = 0x00100000`,

`include/sync0rw.h:54`). Contended acquisitions spin (`SYNC_SPIN_ROUNDS`) and then park in the global **sync array** (`sync/sync0arr.c`), which also powers the "long semaphore wait" diagnostics that InnoDB still prints today.

With hundreds of threads latching thousands of pages, deadlock between latches is prevented not by detection but by **discipline**: a global partial order of latch levels (`SYNC_*` constants, `include/sync0sync.h:421-495`). A thread may only acquire a latch at a *lower* level than every latch it already holds. Excerpts, high to low:

<code>SYNC_DICT (1000)</code>	dictionary
<code>SYNC_INDEX_TREE (900)</code>	index->lock (the tree latch)
<code>SYNC_TREE_NODE (890)</code>	B+tree page latch
<code>SYNC_TRX_UNDO (700) ... SYNC_RSEG (600)</code>	undo/rollback segments
<code>SYNC_FSP (400)</code>	space management
<code>SYNC_KERNEL (300)</code>	the big kernel mutex (trx/lock state, Ch. 7-8)
<code>SYNC_LOG (170)</code>	log mutex
<code>SYNC_BUF_POOL (150)</code>	buffer pool mutex

Debug builds verify every acquisition against the thread's held-latch list (`sync_thread_add_level`, `include/sync0sync.h:214`). This table *is* the concurrency design document of InnoDB: reading it top-to-bottom tells you which subsystem is allowed to call into which. Note that the index tree latch (900) sits above page latches (890) — the B+tree descent in Chapter 6 follows exactly that order.

4.5 What to remember

1. An mtr = **latches + redo records, released/published atomically at commit**. It is the quantum of physical change in InnoDB; user transactions (Chapter 7) are built from many mtrs plus undo logging.
2. `MLOG_MULTI_REC_END` / `MLOG_SINGLE_REC_FLAG` give multi-page atomicity through the log — no shadow paging, no page images.
3. Physiological logging replays runtime code paths during recovery; every page-modifying function has a log-writer and a log-parser twin.
4. Latch deadlocks are prevented by a static latch order (`sync0sync.h`); that ordering table explains the shape of many algorithms in the chapters ahead (e.g. why B+tree code "releases and re-searches" instead of latching upward).

Try it: `grep -c "mtr_t\*" include/*.h` — count how many APIs thread an mtr through; then read `mtr_commit` (`mtr/mtr0mtr.c:176`) side-by-side with section 4.2.

Previous: Chapter 3 — The Buffer Pool · **Next:** Chapter 5 — Redo Log & Crash Recovery

Chapter 5 — The Redo Log & Crash Recovery

Layer 3 of 5 — Durability. How committed work survives any crash: the write-ahead log, checkpoints, and the redo-apply recovery algorithm. Source: `log/log0log.c`, `log/log0recv.c`, `include/log0log.h`, `include/log0recv.h` (note: `log/log0log.c` and `log/log0recv.c` are deleted in this working tree — read them with `git show HEAD:log/log0log.c`)

5.1 The LSN: one number to rule them all

The **log sequence number** is a 64-bit byte offset into the conceptually infinite stream of redo records ever written (starting at `LOG_START_LSN = 8192`, `include/log0log.h:601`). It is InnoDB's global clock. The same unit measures:

- how much log exists (`log_sys->lsn`),
- how much is durably on disk (`flushed_to_disk_lsn`),
- how new a page is (`FIL_PAGE_LSN` on every page, Chapter 2),
- how dirty the buffer pool is (each page's `oldest_modification`, Chapter 3),
- where recovery must start (the checkpoint LSN).

Because everything is measured on one axis, correctness conditions become simple inequalities — e.g. WAL is literally `flushed_to_disk_lsn ≥ page.newest_modification` before a page write.

5.2 The log on disk: blocks, files, checkpoint slots

Redo is written in **512-byte blocks** (`OS_FILE_LOG_BLOCK_SIZE`, `include/os0file.h:109`) — sized to the disk sector, so a block write is assumed atomic (the log's own answer to torn writes; data pages needed the doublewrite buffer instead):

one 512-byte log block

header (12B)	log record data (496 B)	trailer
block no		checksum
data length		(4B)
1st rec grp		
checkpoint no		

(`LOG_BLOCK_*`, `include/log0log.h:606-643`.) `LOG_BLOCK_FIRST_REC_GROUP` marks where the first mtr record-group *starts* in this block — recovery needs it because an mtr's records can straddle blocks.

The log files (`ib_logfile0`, `ib_logfile1`, ...) form one **circular** log group: blocks are written round-robin across files, wrapping to the first file when the last is full (`log_group_write_buf`, `log0log.c:1228`, wraps at `:1277`). Each file starts with a 2048-byte header; the *first* file additionally holds two **checkpoint slots** at offsets 512 (`LOG_CHECKPOINT_1`) and 1536 (`LOG_CHECKPOINT_2`) (`include/log0log.h:679-717`).

Why two slots? Checkpoints alternate between them (`next_checkpoint_no & 1`, `log0log.c:1849-1853`). If the machine dies while writing one, the other still holds a valid older checkpoint — recovery picks whichever slot has a valid checksum and the higher checkpoint number (`recv_find_max_checkpoint`, `log0recv.c:625`).

5.3 Runtime: buffer, group commit, and the WAL rule

In memory, `log_sys` (`include/log0log.h:777-976`) holds a circular **log buffer** and a family of LSN watermarks (`buf_next_to_write`, `write_lsn`, `flushed_to_disk_lsn`, `last_checkpoint_lsn`). Mini-transactions append to the buffer at `mtr_commit` (Chapter 4); `log_write_up_to(lsn, wait, flush_to_disk)` (`log0log.c:1349`) pushes the buffer to disk when someone needs durability up to `lsn`.

`log_write_up_to` contains one of the great old tricks: **group commit** (`log0log.c:1408-1424`). If a write is already in flight and will cover your LSN, you just wait for it; if not, you wait for the current write to finish, then issue one write covering *everyone* who queued up meanwhile. *N* transactions committing concurrently cost ~ 1 fsync, not *N*.

Who calls it, and when:

- **Transaction commit** (`trx_commit_off_kernel`, `trx/trx0trx.c:901-922`) — behavior depends on `srv_flush_log_at_trx_commit`: `1` = write + fsync (real durability), `2` = write to OS only, `0` = do nothing now (the master thread flushes $\sim$ once per second). Yes — this is the original `innodb_flush_log_at_trx_commit`.
- **Page flush** (`buf_flush_write_block_low`, `buf/buf0flu.c:771`) — the WAL rule from Chapter 3.
- **Log-space pressure** — see next section.

5.4 Checkpoints: bounding recovery and reclaiming log space

The log is circular, so old log must eventually be overwritten. A record at LSN *X* may be overwritten only when it is no longer needed — i.e. when every page change it describes has been flushed to disk. The **checkpoint LSN** is exactly that point:

```
checkpoint_lsn = min(oldest_modification) over all dirty pages in the flush list
```

`log_checkpoint()` (`log0log.c:1982`) computes this, first forces `log_write_up_to(oldest_lsn)` (WAL again — `log0log.c:2024-2033`), then writes the checkpoint record into the alternating slot. After that, all log before `checkpoint_lsn` is dead, and recovery may start from `checkpoint_lsn` instead of

from the beginning of time.

This creates the engine's most important feedback loop. The distance `log_sys->lsn - last_checkpoint_lsn` (the "checkpoint age") must stay below the log capacity. Thresholds computed at startup (`log_calc_max_ages`, `log0log.c:671`) drive escalating responses, checked by every operation via `log_free_check()` (`include/log0log.ic:442`):

```
checkpoint age → [ ok ] → [async: background-flush dirty pages, checkpoint ]
                        → [sync: STALL writers until pages flushed          ]
```

This is why "too-small redo logs" make InnoDB slow: small logs → small max age → constant forced flushing and, at worst, synchronous stalls. A tuning fact DBAs learned for decades, visible here as ~30 lines of margin arithmetic.

5.5 Crash recovery

Startup always runs recovery (`recv_recovery_from_checkpoint_start`, `log0recv.c:2927`, called from `srv/srv0start.c:1645`); on a clean shutdown it simply finds nothing to do. Three phases:

1. FIND CHECKPOINT
read both slots, pick valid
one with
highest number
(recv_find_max_checkpoint)



2. SCAN + PARSE from
checkpoint_lsn
(recv_scan_log_recs :2505)
verify each block's number
+ checksum;
a mismatch = torn tail =
end of log.
Parse records
(recv_parse_log_recs
:2183);

keep only COMPLETE mtr
groups
(ending in
MLOG_MULTI_REC_END)

```
graph TD; A[keep only COMPLETE mtr groups  
(ending in MLOG_MULTI_REC_END)] --> B[3. BUILD HASH TABLE  
recv_addr per (space, page_no),  
each holding its list of redo records  
(recv_add_to_hash_table :1311)]; B --> C[4. APPLY  
(recv apply hashed log recs)];
```

3. BUILD HASH TABLE
recv_addr per (space,
page_no),
each holding its list of redo
records
(recv_add_to_hash_table
:1311)

4. APPLY
(recv apply hashed log recs)

:1699)
for each page: read it, then
apply records
in LSN order — idempotently



5. UNDO PHASE (Ch. 7)
roll back transactions that were
active at the crash
(`trx_rollback_or_clean_all_recovered`)

Two details carry most of the intellectual weight:

Idempotent apply. For each record on a page, recovery applies it only if `recv->start_lsn >= page_lsn` — the LSN stored *in the page itself* (`recv_recover_page`, `log0recv.c:1558`). If the page already contains that change (it was flushed before the crash), the record is skipped; after applying, the page LSN is advanced (`:1583-1592`). Crash during recovery? Run recovery again; same rule, same result. The page LSN written at flush time (Chapter 3) and the record LSNs meet here — this comparison is the reason both exist.

The hash table makes redo I/O-efficient. Records are grouped per page, so each page is read once and receives all its changes together (`recv_read_in_area` batches the reads); without this, replaying a large log would re-read hot pages thousands of times. If the hash table outgrows memory, recovery applies-and-clears in batches (`log0recv.c:2505`'s call to `recv_apply_hashed_log_recs(FALSE)`).

After redo, the database is exactly as it was at the instant of the crash — *including* uncommitted changes. Redo is deliberately ignorant of transactions; it restores pages, not semantics. The undo phase (Chapter 7) then rolls back the transactions that never committed, using the undo logs that redo just faithfully reconstructed. Notice the layering: **undo data is itself redo-protected** — that is what makes the whole scheme composable.

5.6 What to remember

1. One LSN axis ties pages, buffer pool, log, and checkpoints together; every correctness rule is an inequality on it.
2. Group commit amortizes fsync across concurrent committers; `flush_log_at_trx_commit` trades durability for speed at that exact point.
3. A checkpoint = "all page changes before LSN X are on disk"; log capacity minus checkpoint age is the engine's breathing room, and running out of it stalls writers.
4. Recovery = redo everything from the checkpoint (idempotently, page-LSN-guarded), then undo the losers. Torn log tails are cut off by block checksums; torn pages were already solved by the doublewrite buffer; incomplete mtrs by `MLOG_MULTI_REC_END`.

Try it: run `tests/.libs/ib_test1`, kill it mid-run (`kill -9`), rerun — and watch the recovery messages InnoDB prints as it scans from the last checkpoint.

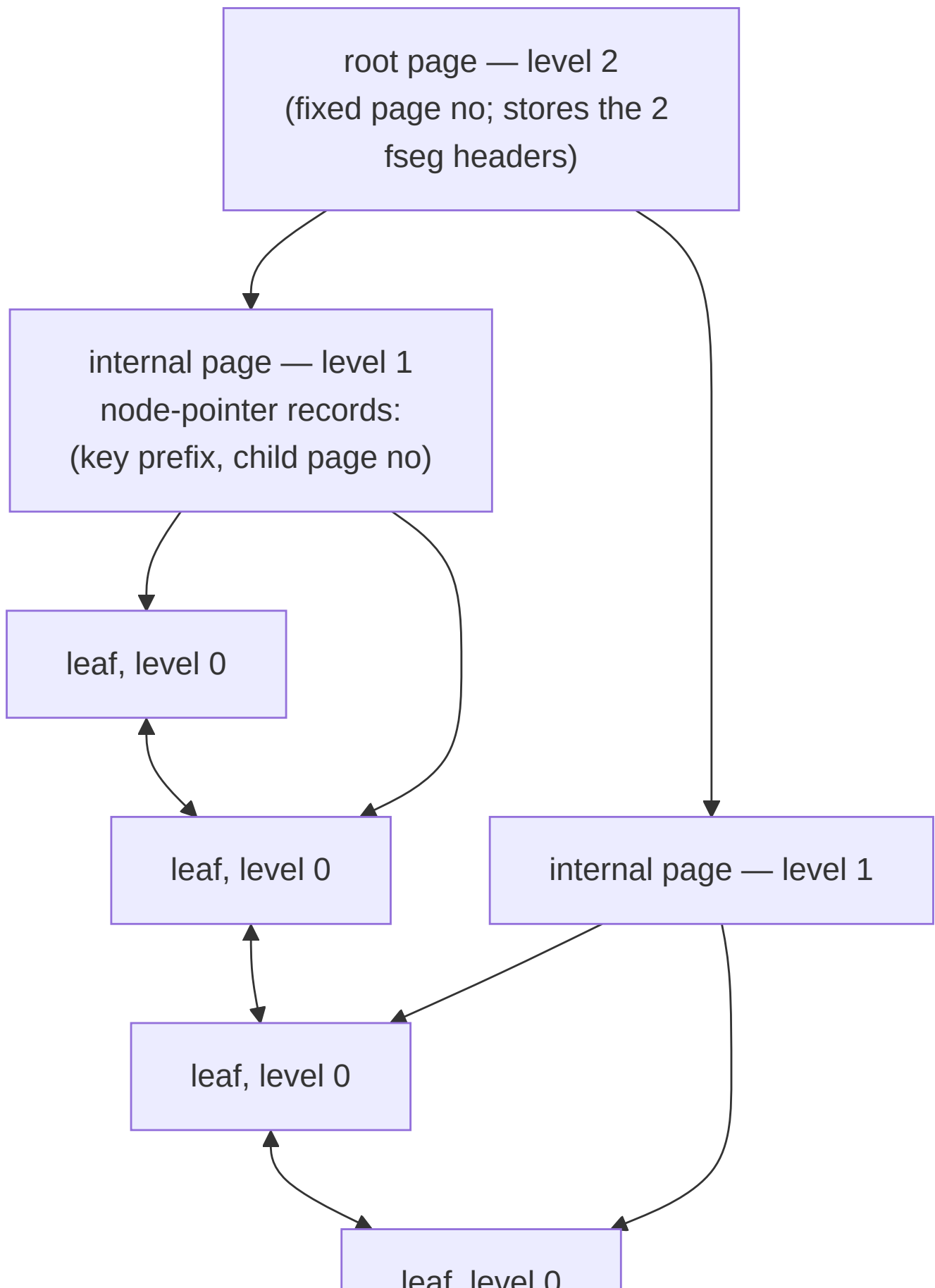
Previous: [Chapter 4 — Mini-Transactions](#) · **Next:** [Chapter 6 — The B+Tree](#)

Chapter 6 — The B+Tree

Layer 4 of 5 — Access paths. How rows are indexed, found, inserted, split, merged — and the adaptive hash shortcut on top. Source: `btr/btr0btr.c`, `btr/btr0cur.c`, `btr/btr0pcur.c`, `btr/btr0sea.c`, `page/page0cur.c`

6.1 Everything is a B+tree

InnoDB has no separate "table storage": **the table *is* its clustered index** — a B+tree keyed on the primary key whose leaf records are the full rows (with the hidden `DB_TRX_ID` / `DB_ROLL_PTR` columns, Chapter 2). Secondary indexes are further B+trees whose leaf records hold the indexed columns + the primary key, so a secondary lookup ends with a second descent into the clustered tree. Even the data dictionary (Chapter 10) and the insert buffer (Chapter 12) are B+trees. Master this chapter and you've mastered most of the engine's disk structure.



Structural facts (`btr/btr0btr.c:46-99` — read this header comment, it's the design doc):

- Internal pages hold **node-pointer records**: a unique key prefix + the 4-byte child page number, exactly one per child. They use the same page/record machinery as leaves (`REC_STATUS_NODE_PTR`, Chapter 2); `PAGE_LEVEL` distinguishes levels, leaf = 0.
- Leaves are **doubly linked** through `FIL_PAGE_PREV/FIL_PAGE_NEXT` — that's what makes range scans a linked-list walk instead of repeated descents.
- The **root page number never changes** (it's stored in the dictionary — `dict_index_t.page`); when the root fills, its contents move *down* into a new child and the root becomes internal (`btr_root_raise_and_insert`, `btr/btr0btr.c:1204`). The tree grows at the top, in place.
- Each index owns **two file segments** (Chapter 1), anchored in the root page: one for non-leaf pages (`PAGE_BTR_SEG_TOP`), one for leaves (`PAGE_BTR_SEG_LEAF`) (`btr_create`, `btr/btr0btr.c:740`) — so leaves cluster together on disk for scan locality.

6.2 Searching: latch crabbing with a tree latch

`btr_cur_search_to_nth_level()` (`btr/btr0cur.c:345`) positions a cursor. The latching scheme (per the `btr0btr.c` header and the `SYNC_*` order from Chapter 4):

```
S-latch index->lock (the tree latch)
descend: root → ... → leaf, each internal page only buffer-fixed (no latch)
latch the leaf (S for read, X for modify)
release the tree latch
```

Readers and point-writers hold the tree latch only during descent. A **restructuring** operation (split/merge) instead takes the tree latch in X mode (`BTR_MODIFY_TREE`), which freezes the tree's *shape* — this is a classic 1990s-style tree-level scheme, simpler but less concurrent than the lock-free/B-link approaches modern InnoDB later adopted. Within a page, positioning is the directory binary search of Chapter 2 (`page_cur_search_with_match`, `page/page0cur.c:254`).

Latch modes tell the story (`include/btr0btr.h:55-82`): `BTR_SEARCH_LEAF` (S), `BTR_MODIFY_LEAF` (X on leaf), `BTR_MODIFY_TREE` (X on tree), plus OR-flags like `BTR_INSERT` ("if the leaf isn't cached, buffer this insert instead" — the insert buffer hook, Chapter 12).

Persistent cursors: surviving latch release

A raw cursor dies when its page latch is released. But row operations must span mini-transactions (e.g. fetch a row, return to the user, continue the scan). The **persistent cursor** (`btr_pcur_t`, `btr/btr0pcur.c`) bridges this:

- `btr_pcur_store_position()` (:89) copies the record's key prefix and remembers the block + its `modify_clock` (Chapter 3).
- `btr_pcur_restore_position()` (:208) first tries the **optimistic** path: if the same block is still in the pool with an unchanged `modify_clock`, jump straight back — no descent. Else re-search from the root using the stored key.

This optimistic-revalidation pattern (version counter + retry) appears all over InnoDB; here is its most important use.

6.3 Inserting: optimistic first, pessimistic if needed

Every insert tries the cheap path before the expensive one (`row_ins_index_entry`, `row/row0ins.c:2160` — flow detailed in Chapter 9):

1. **Optimistic** (`btr_cur_optimistic_insert`, `btr/btr0cur.c:1059`): descend with `BTR_MODIFY_LEAF`, X-latch just the leaf; if the record fits, splice it in (`page_cur_insert_rec_low`, `page/page0cur.c:966` — reuse a slot from the page free list or extend the heap, then link into the key-order list). The overwhelmingly common case: one page latch, no tree latch.
2. **Pessimistic** (`btr_cur_pessimistic_insert`, `btr/btr0cur.c:1341`): the page was full. X-latch the tree, then split.

Page splits — and the sequential-insert optimization

`btr_page_split_and_insert()` (`btr/btr0btr.c:1885`) picks the split point (:1941-1991):

- Default: split at the **middle record** — classic 50/50 B-tree behavior, good for random inserts.
- But if `PAGE_LAST_INSERT` shows the new record continues an ascending run (`btr_page_get_split_rec_to_right`, :1427), split at the *insertion point*, keeping the full page intact and starting a nearly-empty new right page.

That heuristic is why monotonically increasing primary keys (auto-increment) fill pages ~100% instead of 50% — and why every MySQL performance guide since has told you to prefer auto-increment PKs over random UUIDs. The page header's `PAGE_DIRECTION/PAGE_N_DIRECTION` fields (Chapter 2) exist to feed exactly this decision. The mirrored descending case is `btr_page_get_split_rec_to_left` (:1382).

After moving records, the split inserts a new node pointer into the parent (`btr_insert_on_non_leaf_level`, :1707) — which may recursively split the parent, up to the root raise. All of it happens inside one mini-transaction: crash anywhere, and Chapter 4's `MLOG_MULTI_REC_END` rule makes the whole split vanish or persist atomically.

Deletes, merges, and BLOBs

- Deletion is two-phase: transactions only **delete-mark** records (`btr_cur_del_mark_set_clust_rec`, `btr/btr0cur.c:2581` — sets the info bit, logs to undo); physical removal is purge's job (Chapters 7, 9).
- When a page's fill drops low, `btr_compress` (`btr/btr0btr.c:2569`) merges it into a sibling; an emptied page is unlinked by `btr_discard_page` (`:2932`). Both need the tree X-latch.
- A record too large for a page ($> \sim\frac{1}{2}$ page, `BTR_PAGE_MAX_REC_SIZE`, `include/btr0btr.h:40`) has its longest columns moved to a chain of BLOB pages (`btr_store_big_rec_extern_fields`, `btr/btr0cur.c:3872`), leaving a 768-byte prefix + a 20-byte external reference in the record — the origin of the famous "768-byte prefix" of MySQL's Antelope row format.

6.4 The adaptive hash index

Descending 3 levels for every point lookup is wasteful when the same keys are probed repeatedly. `btr/btr0sea.c` watches search patterns per page and, when a page has served enough (~ 100 , `BTR_SEARCH_BUILD_LIMIT`, `btr0sea.c:92`) hash-friendly lookups, builds an in-memory **hash index** over its records: `hash(key prefix) → record pointer`, bypassing the descent entirely (`btr_search_guess_on_hash`, `btr0sea.c:835`).

It's a pure cache — built lazily, dropped when the page is evicted, protected by one global `btr_search_latch` (a scalability pain point then and for the next fifteen years of InnoDB history; the tuning knob `adaptive_hash_index` exists in this codebase's API, `api/api0cfg.c:649`). Every page-modifying operation in `btr0cur.c` dutifully updates or invalidates it — `grep btr_search_update_hash` to see the maintenance burden a "simple cache" imposes.

6.5 What to remember

1. Table = clustered B+tree; secondary index = B+tree pointing at primary keys. One structure, used everywhere (dictionary, ibuf, undo... everything but the log).
2. Concurrency = tree latch for shape + page latches for content, ordered per Chapter 4; positions survive latch release via persistent cursors and `modify_clock`.
3. Writes are **optimistic-then-pessimistic**; splits are logged as atomic mtrs; the split-point heuristic rewards sequential keys — a design decision you can still feel in every MySQL schema review.
4. Delete = mark now, purge later (the B+tree meets MVCC in the next chapter).
5. The adaptive hash index shows the cost/benefit of transparent caching: free speed on skewed point reads, a global latch and invalidation logic everywhere else.

Try it: `tests/.libs/ib_index` exercises index creation; trace a split by breaking on `btr_page_split_and_insert` in `tests/debug_gdb.sh` while `ib_bulk_insert` runs.

Previous: [Chapter 5 — Redo Log & Recovery](#) · **Next:** [Chapter 7 — Transactions, Undo & MVCC](#)

Chapter 7 — Transactions, Undo Logs & MVCC

Layer 5 of 5 — Transactions. Commit, rollback, and the trick that lets readers never block writers: multi-version concurrency control built on undo logs. Source: `trx/trx0trx.c`, `trx/trx0sys.c`, `trx/trx0undo.c`, `trx/trx0rec.c`, `trx/trx0roll.c`, `trx/trx0purge.c`, `read/read0read.c`, `row/row0vers.c`

7.1 The transaction object and system

A transaction is a `trx_t` (`include/trx0trx.h:463-696`). The fields that matter most:

field	meaning
<code>id</code>	assigned at start, from a global counter
<code>no</code>	<i>serialization number</i> , assigned at commit — commit order
<code>conc_state</code>	<code>TRX_NOT_STARTED</code> → <code>TRX_ACTIVE</code> → (<code>TRX_PREPARED</code>) → <code>TRX_COMMITTED_IN_MEMORY</code>
<code>isolation_level</code>	<code>TRX_ISO_*</code> ; default REPEATABLE READ (<code>include/trx0trx.h:717-749</code>)
<code>read_view</code>	the MVCC snapshot (below)
<code>insert_undo</code> , <code>update_undo</code>	the trx's two undo logs
<code>undo_no</code>	count of undo records written — the rollback "stack pointer"

Shared state lives in `trx_sys` (`include/trx0sys.h:527-554`): the list of open transactions, the `max_trx_id` counter, the rollback segments, and the read views. Almost all of it is protected by one giant **kernel mutex** (`SYNC_KERNEL`, Chapter 4) — the coarse-grained choice that later versions of InnoDB spent a decade breaking apart.

Persistence lives on the **TRX_SYS page** — page 5 of the system tablespace (`TRX_SYS_PAGE_NO`, `include/trx0sys.h:402`): the high-water `max_trx_id` (written every 256 ids so a restart can never reuse one, `TRX_SYS_TRX_ID_WRITE_MARGIN`, `include/trx0sys.h:559`), 256 rollback-segment slots, and — as seen in Chapter 3 — the doublewrite buffer's location.

7.2 Undo logs: the before-image store

Redo (Chapters 4-5) makes changes durable; **undo makes them revocable and multi-versioned**. Undo records live in ordinary 16KB pages (`FIL_PAGE_UNDO_LOG`) inside **rollback segments** — so they are buffered, latched, and *redo-logged* like everything else.

TRX_SYS page (space 0, page 5)

- └ 256 rollback segment slots → rseg header page (page 6 = the first)
 - └ rseg: TRX_RSEG_UNDO_SLOTS (1024 slots) + TRX_RSEG_HISTORY list
 - └ undo log segment (one per active trx & type)
 - └ undo pages → undo records

Each transaction that writes gets a rollback segment (round-robin, `trx_assign_rseg`, `trx/trx0trx.c:608`) and up to two undo logs in it (`include/trx0undo.h:346-360`):

- **insert undo** — records for freshly inserted rows. After commit these are worthless (the row is fully visible; no older version exists), so they're freed immediately.
- **update undo** — records for updates and delete-marks: the **before image**. These must outlive commit, because other transactions' snapshots may still need the old version.

An update-undo record (`trx/trx0rec.c`; types `TRX_UNDO_UPD_EXIST_REC` etc., `include/trx0rec.h:309-330`) stores: table id, the primary key of the row, the old `DB_TRX_ID` + `DB_ROLL_PTR`, and the old values of changed columns.

The version chain

Recall the hidden columns (Chapter 2). Every update rewrites them:

```
row (current, in clustered index leaf)
  DB_TRX_ID = 105, DB_ROLL_PTR → undo rec (before image, trx 105)
                                old DB_TRX_ID = 92, old DB_ROLL_PTR → undo rec (trx 92)
                                old DB_TRX_ID =
71 ...
```

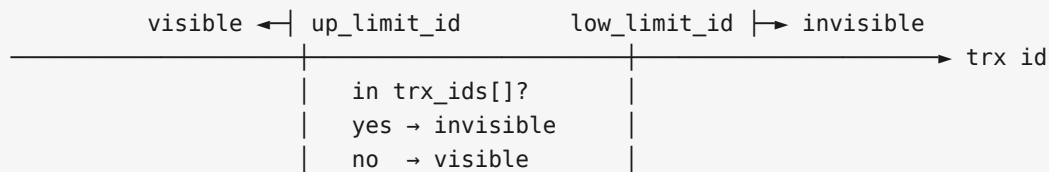
`DB_ROLL_PTR` is a packed 7-byte (rseg id, page no, offset) pointer (`trx_undo_build_roll_ptr`, `include/trx0undo.ic:35`), with its top bit flagging insert-vs-update undo. Follow the chain and you time-travel backwards through the row's history. **This chain is the entire basis of MVCC.**

7.3 Read views: deciding what a transaction may see

A **read view** (`read_view_t`, `include/read0read.h:126-162`) is a snapshot of the transaction state at one instant, built by `read_view_open_now()` (`read/read0read.c:252`) under the kernel mutex:

- `low_limit_id` — `max_trx_id` at snapshot time: any `trx id` $\geq$ this started *after* the snapshot → invisible.
- `up_limit_id` — the smallest active id: any `trx id` $<$ this had committed *before* the snapshot → visible.
- `trx_ids[]` — ids active (or prepared) at snapshot time → invisible (in-flight).

Visibility of a row version is then `read_view_sees_trx_id(view, DB_TRX_ID)` (`include/read0read.ic:61-98`):



When a reader lands on a record whose `DB_TRX_ID` it cannot see, it rebuilds an older version: `row_vers_build_for_consistent_read()` (`row/row0vers.c:484`) walks the `DB_ROLL_PTR` chain, applying before-images one step at a time, until it reaches a version the view sees (or the chain's start — meaning the row didn't exist for this reader). **Readers take no locks and block nothing;** they pay with CPU and undo-page reads proportional to how far behind they are.

Isolation levels fall out of *when the view is created*:

- **REPEATABLE READ** (default): one view per transaction, created at the first read (`trx_assign_read_view`, `trx/trx0trx.c:976`) and reused — every statement sees the same snapshot.
- **READ COMMITTED**: the view is closed after each statement (`read_view_close_for_read_committed`, `read/read0read.c:340`), so the next statement builds a fresh one — it sees everything committed so far.
- **READ UNCOMMITTED**: skip version building entirely (read the newest version, dirty).
- **SERIALIZABLE**: reads become locking reads (Chapter 8); MVCC steps aside.

7.4 Commit and rollback

Commit is cheap by design

`trx_commit_off_kernel()` (`trx/trx0trx.c:730`) does **no work proportional to the transaction's size** — the changes are already in the pages, already redo-logged:

1. In one mini-transaction: mark the undo logs finished, assign the serialization number `trx->no`, and hand the update-undo log to the **history list** of its rollback segment (`trx_undo_update_cleanup` → `trx_purge_add_update_undo_to_history`, `trx/trx0purge.c:308`). The mtr's commit is *the* durable commit point.
2. Set `conc_state = TRX_COMMITTED_IN_MEMORY` and release all locks (`lock_release_off_kernel`, Chapter 8).
3. Flush the log up to the commit LSN per `flush_log_at_trx_commit` (Chapter 5's group commit).

Note what makes this correct: after step 1, crash recovery will find the undo log in "finished" state and treat the transaction as committed; before it, recovery rolls it back. A transaction's fate is decided by one atomic mtr on one undo page.

Rollback replays undo in reverse

Rollback (`trx_general_rollback`, `trx/trx0roll.c:66`) pops undo records in strictly descending `undo_no` order across both undo logs (`trx_roll_pop_top_rec_of_trx`, `trx/trx0roll.c:677`) and dispatches each to the row layer (`row_undo`, `row/row0undo.c:232`): an insert-undo record removes the inserted row; an update-undo record restores the before image. Because `undo_no` counts records per transaction, **partial rollback is free**: rolling back to a savepoint (`trx_savept_take`, `trx/trx0roll.c:173`) just stops popping at the saved `undo_no` — the same mechanism rolls back a single failed statement (`trx->last_sql_stat_start`).

Crash recovery's undo phase (Chapter 5) is this very code, run against transactions found `TRX_ACTIVE` after redo. And XA two-phase commit (`trx_prepare_off_kernel`, `trx/trx0trx.c:1792`) is a small variation: persist the state `TRX_PREPARED` (+XID) in the undo header, flush the log, and let the coordinator decide later (`trx_recover`, :1930, lists in-doubt transactions after a crash).

7.5 Purge: taking out the MVCC garbage

Two kinds of garbage accumulate: update-undo logs of committed transactions (the history list), and delete-marked records still physically sitting in indexes (Chapter 6). Neither can be removed while *any* read view might still need them.

The **purge system** (`trx_purge_t`, `include/trx0purge.h:131-186`, driven from the master thread — Chapter 12) computes its horizon the elegant way: it *clones the oldest existing read view* (`read_view_oldest_copy_or_open_new`, `read/read0read.c:168`). Whatever is invisible to that view is invisible to everyone — safe to destroy. Purge then walks the history lists in commit order (`trx_purge`, `trx/trx0purge.c:1099`), physically removing delete-marked records from indexes (`row/row0purge.c`, Chapter 9) and freeing undo pages (`trx_purge_truncate_history`, :599).

The failure mode is famous: **a long-running read view stalls purge**, the history list grows (`trx_sys->rseg_history_len`), version chains lengthen, reads slow down. This codebase already has the pressure valve — `max_purge_lag` (`srv_max_purge_lag`, checked in `trx_purge` :1137-1151), which artificially delays DML when history grows too long. Twenty years later, "HISTORY LIST LENGTH" is still the first thing a DBA checks on a struggling MySQL server.

7.6 What to remember

1. Undo = before-images in regular, redo-protected pages; each row heads a **version chain** through `DB_ROLL_PTR`.
2. A read view is three numbers and a list; visibility is a comparison against them, and isolation levels differ only in *when views are (re)built*.
3. Commit is O(1): flip undo state, assign commit number, release locks, flush log. Rollback is O(changes): pop undo records in reverse. Recovery reuses both.
4. Purge is a garbage collector whose horizon is the oldest read view; the history list is its backlog gauge.

Try it: `tests/.libs/lib_mt_stress` runs concurrent transactions; break in `read_view_open_now` and inspect `trx_ids` to watch a snapshot being taken.

Previous: [Chapter 6 — The B+Tree](#) · **Next:** [Chapter 8 — The Lock Manager](#)

Chapter 8 — The Lock Manager

Layer 5 of 5 — Transactions. How writers coordinate: row and table locks, gap locks against phantoms, and deadlock detection. Source: `lock/lock0lock.c`, `include/lock0lock.h`, `include/lock0priv.h`

8.1 Locks vs latches — don't confuse them

	latch (Ch. 4)	lock (this chapter)
protects	in-memory page/structure	logical data (rows, tables)
held for	microseconds (one mtr)	until transaction commit
deadlocks	prevented by static order	detected and resolved
module	<code>sync/</code>	<code>lock/</code>

MVCC (Chapter 7) removed *read-write* conflicts. The lock manager handles what remains: **write-write conflicts** and **locking reads** (SELECT ... FOR UPDATE, foreign-key checks, SERIALIZABLE).

8.2 Modes and intention locking

Lock modes (`enum lock_mode`, `include/lock0types.h:34`): `LOCK_S`, `LOCK_X`, plus table-level `LOCK_IS`, `LOCK_IX` and `LOCK_AUTO_INC`. Compatibility (`lock/lock0lock.c:290-321`):

	IS	IX	S	X	AI
IS	✓	✓	✓	x	✓
IX	✓	✓	x	x	x
S	✓	x	✓	x	x
X	x	x	x	x	x
AI	✓	x	x	x	x

Intention locks solve a hierarchy problem: to lock the whole table (e.g. for DROP), you must know no one holds row locks inside it — without scanning every row lock. So every row lock is preceded by an IS/IX lock *on the table*; a table S/X request then conflicts with IS/IX directly, in O(1). `LOCK_AUTO_INC` is the special table lock serializing auto-increment allocation.

8.3 Record locks: bitmaps and gap semantics

Storage: one lock object covers a page

A naive row lock = one heap object per (trx, row) — millions of objects for a big update. InnoDB's representation (`lock_rec_create_low`, `lock/lock0lock.c:1684`):

```
lock_t { trx, type_mode, index, {space, page_no, n_bits} }  
        followed immediately in memory by a BITMAP, one bit per heap_no on that page
```

One `lock_t` per (transaction, page, mode) covers *any number of rows on that page* — a bit per record heap number (Chapter 2's `heap_no`). All record locks live in one global hash table keyed by (space, page_no) (`lock_sys->rec_hash`, `include/lock0lock.h:814`). Locking a row = find/create the lock object, set a bit. This is why InnoDB row locks are so cheap (~1 bit amortized) — and why lock objects must be reorganized when records move between pages (`lock_update_*` calls sprinkled through `btr0btr.c`).

Gap locks: the phantom problem

Between committed rows there are **gaps**, and a plain row lock says nothing about them: transaction A locks "all rows WHERE k BETWEEN 10 AND 20", transaction B *inserts* k=15, A re-reads and sees a phantom. InnoDB's REPEATABLE READ kills phantoms with three record-lock flavors (`include/lock0lock.h:752-803`):

```
index:      ... [k=10] —gap— [k=20] ...  
  
LOCK_REC_NOT_GAP  locks the record only          (unique-key point lookups)  
LOCK_GAP          locks the gap before it only (nothing may be INSERTed there)  
LOCK_ORDINARY     "next-key" = record + gap    (range scans; the default)
```

A scan under locking-read acquires next-key locks on every record it passes: rows can't change, gaps can't be filled — the range is sealed. The supremum record (Chapter 2) receives the lock for "the gap after the last row". Inserts negotiate via `LOCK_INSERT_INTENTION` (:789): an insert into a locked gap enqueues this special waiting gap-lock; it conflicts with gap/next-key locks (so range scanners block phantom inserts) but two inserts into the same gap at different positions don't block each other.

READ COMMITTED, by contrast, skips gap locks (`row/row0sel.c:3167`) — phantoms allowed, concurrency higher: the exact trade-off MySQL users pick with `transaction_isolation` today.

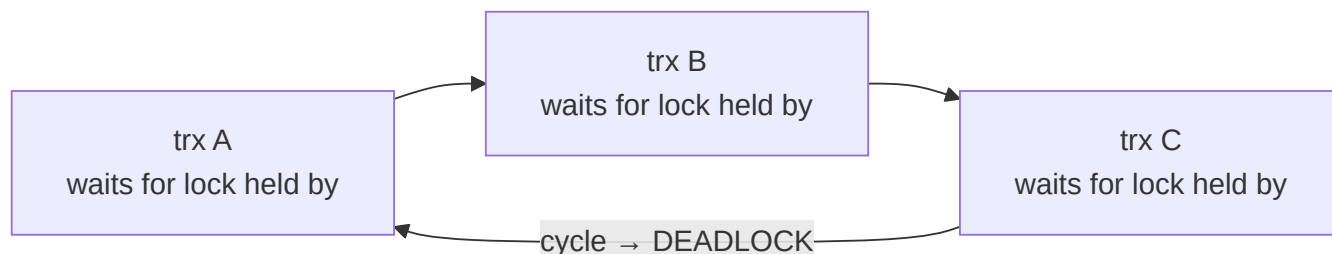
There is also a hidden economy: a freshly inserted, uncommitted row is protected not by any lock object but **implicitly** — its `DB_TRX_ID` names an active transaction. Only when a second transaction actually conflicts does the lock manager materialize an explicit lock on the first one's behalf (`row_vers_impl_x_locked_off_kernel`, `row/row0vers.c:56`). Most locks are never created at all.

8.4 Waiting, timeouts, deadlocks

All lock-manager state is guarded by the kernel mutex (Chapter 7's big lock). A conflicting request enqueues a lock with `LOCK_WAIT` set (`lock_rec_enqueue_waiting`, `lock/lock0lock.c:~1795`), the transaction goes to `TRX_QUE_LOCK_WAIT`, and its thread sleeps. Grants happen when the blocker releases: `lock_rec_dequeue_from_page` (:2268) scans the queue and grants whatever is now compatible — FIFO-ish, preventing starvation. A background thread times out waits (`srv_lock_timeout_thread`, Chapter 12; tunable `lock_wait_timeout`).

Deadlock detection: DFS on the wait-for graph

Because locks are held to commit and acquired incrementally, deadlocks are inevitable. At each new *wait*, InnoDB runs `lock_deadlock_occurs()` → `lock_deadlock_recursive()` (`lock/lock0lock.c:3292`, :3373) — a depth-first search along "who is waiting for whom":



- Search cost is bounded: depth ≤ 200 , steps $\leq 1,000,000$ (`LOCK_MAX_DEPTH_IN_DEADLOCK_CHECK`, `LOCK_MAX_N_STEPS_IN_DEADLOCK_CHECK`, `lock/lock0lock.c:44-48`); beyond that, assume the worst and abort the requester — a pragmatic answer to graph-search blowup.
- On a cycle, the **victim** is the transaction with less weight (`trx_weight_cmp` — roughly: fewer undo records + fewer locks = cheaper to roll back); it gets `DB_DEADLOCK` and is rolled back (Chapter 7's machinery).

Commit releases everything: `lock_release_off_kernel()` (`lock/lock0lock.c:3992`) walks the trx's lock list, granting queued waiters lock by lock, then discards the whole lock heap in one free.

8.5 What to remember

1. Latches protect memory for microseconds; locks protect data until commit. MVCC removed reader-writer conflicts; the lock manager arbitrates writers.
2. Row locks are **bitmaps hanging off (page, trx, mode) objects** in a page-keyed hash — the representation is the scalability trick. Implicit locks avoid even that until a conflict materializes.
3. **Next-key = record + preceding gap**; gap locks + insert-intention locks are how REPEATABLE READ prevents phantoms without predicate locks. Supremum owns the last gap.

4. Deadlock handling = bounded DFS at wait time + cheapest-victim rollback; lock waits and timeouts are the visible symptoms (`ib_deadlock.c` in tests/ demonstrates one).

Try it: `tests/ib_deadlock.c` constructs a deadlock; run it and watch one transaction get `DB_DEADLOCK`. Break in `lock_deadlock_recursive` to watch the DFS walk the wait-for graph.

Previous: [Chapter 7 — Transactions & MVCC](#) · **Next:** [Chapter 9 — Row Operations](#)

Chapter 9 — Row Operations: INSERT, SELECT, UPDATE, DELETE End-to-End

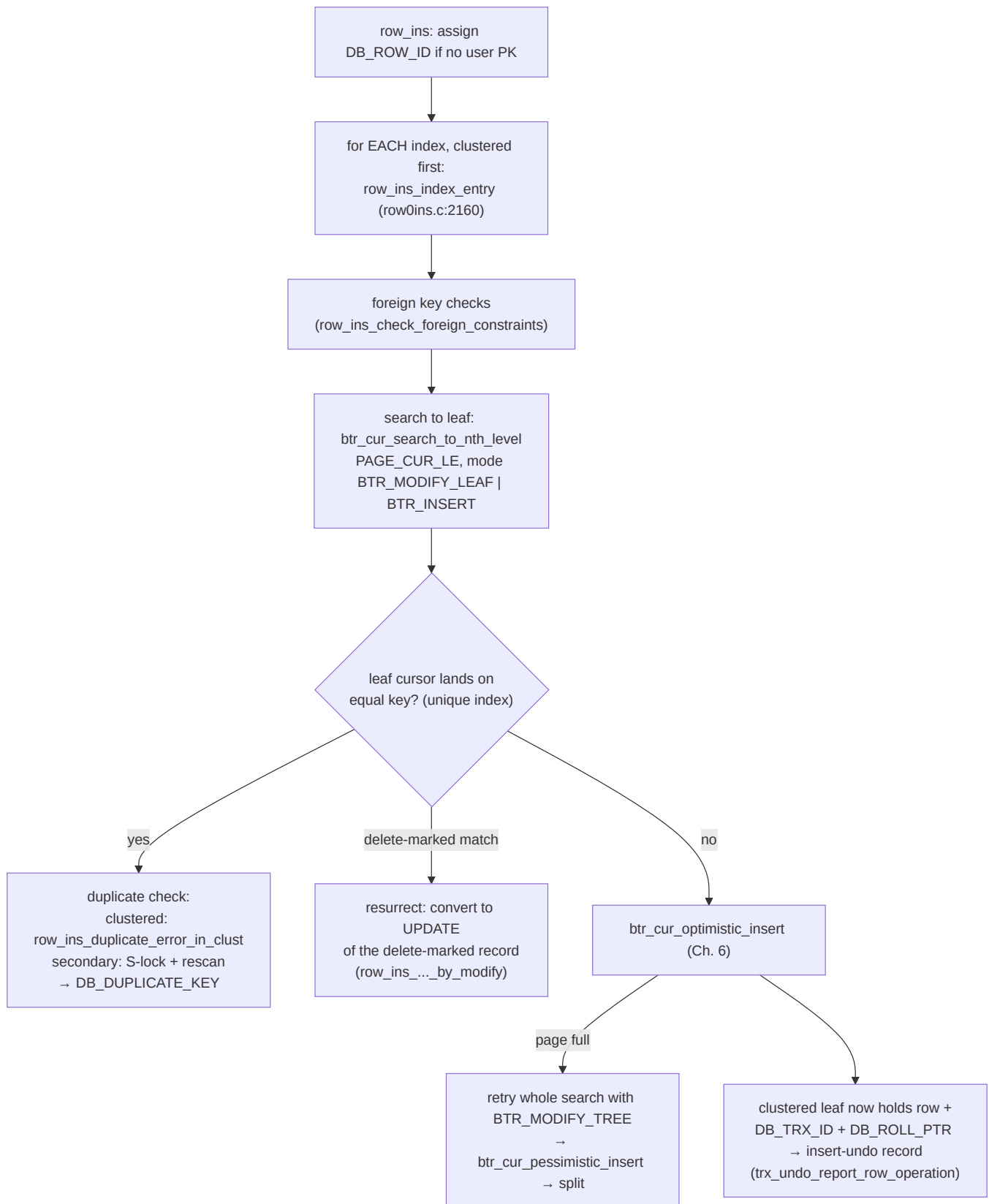
Layer 4-5 glue. The `row/` module composes everything from Chapters 1-8 into the four verbs of a database. This chapter follows each verb all the way down. Source: `row/row0ins.c`, `row/row0sel.c`, `row/row0upd.c`, `row/row0undo.c`, `row/row0purge.c`, `row/row0row.c`, `row/row0merge.c`

9.1 The cast of characters

- **`row_prebuilt_t`** (`row/row0prebuilt.c`) — a per-cursor cache of everything needed to run row operations on one table (index handles, a persistent cursor, heaps, the search tuple). The embedded API's `ib_csr_t` wraps one (`api/api0api.c:2866`).
- **Query graphs** — row operations run as nodes (`ins_node_t`, `upd_node_t`, `sel_node_t`) stepped by InnoDB's internal executor (`que/que0que.c`, Chapter 10). The API builds a tiny graph and pumps it (`row_ins_step`, `row_upd_step`); only SELECT has a direct entry point.
- **`row_build*` helpers** (`row/row0row.c`) — convert between physical records and in-memory tuples: `row_build` (record → row), `row_build_index_entry` (row → entry for index N), `row_build_row_ref` (secondary record → primary key).

9.2 INSERT

Full descent from the API (`api/api0api.c:3353` → `row_ins_step` → `row_ins`, `row/row0ins.c:2375`):



Things worth noticing:

- **Optimistic-then-pessimistic appears again** (`row0ins.c:2183-2193`) — the same two-attempt pattern as the B+tree layer, one level up.
- **Insert doesn't lock the new row.** It relies on the implicit lock (Chapter 8): the row's own `DB_TRX_ID` protects it. It *does* wait on insert-intention vs gap locks if someone has the gap sealed.
- **A delete-marked duplicate is an update in disguise** (`row_ins_must_modify`): if the same key was deleted (but not yet purged, Chapter 7), the "insert" reuses the tombstone record. MVCC's deferred deletion leaks into insert logic — a recurring price you'll now recognize.
- If the leaf page isn't in the buffer pool and this is a non-unique secondary index, the `BTR_INSERT` flag routes the entry into the **insert buffer** instead (Chapter 12) — the insert completes without a single random read.

9.3 SELECT

The embedded API's row reader is `row_search_for_client()` (`row/row0sel.c:2832`) — a ~700-line function that is the direct ancestor of MySQL's `row_search_for_mysql` / `row_search_mvcc`. Skeleton:

1. Restore the persistent cursor (or search from the tuple) at `BTR_SEARCH_LEAF`.
2. For each candidate record:
 - **Consistent read** (default): check `read_view_sees_trx_id(DB_TRX_ID)`; if not visible, rebuild the old version via the undo chain (`row_vers_build_for_consistent_read`, Chapter 7) — a secondary-index record has no `DB_TRX_ID`, so if the page's `PAGE_MAX_TRX_ID` is too new, fall through to the clustered record to decide visibility (`row_sel_get_clust_rec_for_client`).
 - **Locking read** (FOR UPDATE / SERIALIZABLE): acquire the record/next-key lock instead (`lock_clust_rec_read_check_and_lock`); may return `DB_LOCK_WAIT` — the row layer suspends the thread and retries after the wait.
 - Skip delete-marked versions that resolve to "row gone".
3. Convert the winning record to the caller's tuple format; store the cursor position for the next fetch (`btr_pcur_store_position`, Chapter 6).

The interplay to appreciate: **visibility (MVCC), locking, and cursor positioning are three independent state machines** braided into one loop — that braid, not B+tree search, is what makes `row0sel.c` the hardest file in the engine.

9.4 UPDATE and DELETE

DELETE in InnoDB *is* an UPDATE (set the delete-mark). Both run through `row_upd_step` → `row_upd` (`row/row0upd.c:2004`), always clustered index first:

```

row_upd_clust_step (row0upd.c:1832)
├ restore cursor on the clustered record (BTR_MODIFY_LEAF)
├ write update-undo record (before image) — gets roll_ptr
├ CASE 1: no ordering (key) column changed
│   └ update in place: btr_cur_update_in_place / optimistic_update
│     (new DB_TRX_ID, DB_ROLL_PTR → version chain grows, Ch. 7)
└ CASE 2: a PK column changed
    └ delete-mark the old record + INSERT a fresh one
      (row_upd_clust_rec_by_insert — the row physically moves!)
then, for each affected secondary index (row_upd_sec_index_entry :1437):
    delete-mark the old entry + insert the new entry ← ALWAYS, never in place

```

Two design consequences that echo into modern MySQL:

- **Changing a primary key is expensive** — it relocates the row and touches every secondary index. ("Never update your PK" is not superstition; it's `row_upd_clust_rec_by_insert`.)
- **Secondary indexes are never updated in place** (`row0upd.c:1437-1526`): they have no version info of their own (no `DB_TRX_ID` per entry), so old and new entries must coexist as delete-marked/live pairs, with readers resolving via the clustered record. Purge cleans up the tombstones later.

9.5 The other half: UNDO and PURGE as row operations

The transaction chapter showed *when* undo and purge run; `row/` implements *what* they do — each is the mirror image of a forward operation:

- **Rollback** (`row/row0undo.c:232` dispatching to `row0uins.c` / `row0umod.c`): an insert-undo record → delete the inserted row (its entries in every index); an update-undo record → restore the before image (and un-mark / re-insert secondary entries).
- **Purge** (`row/row0purge.c:599`): for a `TRX_UNDO_DEL_MARK_REC`, physically remove the delete-marked entries — secondary indexes first, clustered last (`row_purge_del_mark`, :346), each with the optimistic/pessimistic pair. Before removing a secondary entry, purge must prove no old version still needs it (`row_vers_old_has_index_entry`, `row/row0vers.c:341`).

So a row's full life is: INSERT (Ch. 9.2) → versions accumulate via UPDATE (9.4 + Ch. 7) → DELETE = mark (9.4) → PURGE = physical removal (9.5). No path removes a record from a leaf synchronously with the user's statement — every one of them defers to keep MVCC readers safe.

9.6 Bonus: fast index creation

`row/row0merge.c` is a fascinating late addition (from the InnoDB plugin era): CREATE INDEX doesn't insert rows one-by-one into the new B+tree (random I/O, page splits). Instead (`row_merge_build_indexes`, :2260): scan the clustered index once → sort entries in 1MB blocks with

an external **merge sort** on temp files → bulk-load the sorted stream into the new tree left-to-right. Sequential I/O, ~full pages, no splits: the same reasoning as the sequential-insert split optimization of Chapter 6, applied wholesale.

9.7 What to remember

1. `row/` is choreography: every verb = dictionary metadata + B+tree cursors + undo records + locks + read views, composed in a fixed order (clustered first, secondaries after).
2. SELECT braids MVCC visibility, locking, and cursor persistence in one loop (`row_search_for_client`) — read it once slowly; everything else in `row/` is easier.
3. UPDATE is in-place only when the key doesn't change; secondary indexes always delete+insert; DELETE only marks. Purge is the one who actually deletes.
4. Undo and purge are row operations too — rollback is the inverse verb, purge is the deferred second half of DELETE.

Try it: `ltrace -e 'ib_*' tests/.libs/lib_test1` shows the API sequence; then break on `row_ins_index_entry` and `row_search_for_client` in gdb to see each verb enter the row layer.

Previous: [Chapter 8 — The Lock Manager](#) · **Next:** [Chapter 10 — The Data Dictionary](#)

Chapter 10 — The Data Dictionary & the Internal SQL Interpreter

Where InnoDB keeps its own schema — in itself — and the miniature SQL engine it uses to do so. Source: `dict/dict0boot.c`, `dict/dict0dict.c`, `dict/dict0load.c`, `dict/dict0crea.c`, `pars/`, `que/`, `eval/`

10.1 The bootstrapping trick

InnoDB stores table metadata *in InnoDB tables*. That's circular: to read a table you need its metadata, which lives in a table. The cycle is broken with one well-known page — the **dictionary header**, page 7 of the system tablespace (`DICT_HDR_PAGE_NO`, `include/dict0boot.h:96`):

DICT_HDR page (space 0, page 7)include/dict0boot.h:118-132

DICT_HDR_ROW_ID	next DB_ROW_ID to assign
DICT_HDR_TABLE_ID	next table id
DICT_HDR_INDEX_ID	next index id
DICT_HDR_TABLES	root page # of SYS_TABLES tree
DICT_HDR_TABLE_IDS	root page # of SYS_TABLE_IDS tree
DICT_HDR_COLUMNS	root page # of SYS_COLUMNS tree
DICT_HDR_INDEXES	root page # of SYS_INDEXES tree
DICT_HDR_FIELDS	root page # of SYS_FIELDS tree

fixed entry points
into four B+trees
whose schemas are
HARD-CODED in C

The four **system tables** are ordinary B+trees; what's special is that their column layouts are compiled into the engine (`dict_boot()`, `dict/dict0boot.c:226`, builds their `dict_table_t` descriptors from literal C code):

table	one row per	key columns
SYS_TABLES	table	NAME → (ID, N_COLS, TYPE, SPACE, ...)
SYS_COLUMNS	column	(TABLE_ID, POS) → (NAME, MTYPE, PRTYPE, LEN, ...)
SYS_INDEXES	index	(TABLE_ID, ID) → (NAME, N_FIELDS, TYPE, SPACE, PAGE_NO)
SYS_FIELDS	index column	(INDEX_ID, POS) → COL_NAME

`SYS_INDEXES.PAGE_NO` is the payoff: it stores each index's **root page number** — the single number that connects a name like `t1(PRIMARY)` to a B+tree you can descend (Chapter 6). At first database creation, `dict_hdr_create()` (`dict/dict0boot.c:124`) `btr_create()`s the four trees and writes their roots into the header; on every later startup `dict_boot()` just reads them back. Bootstrap complete.

10.2 The in-memory cache

Parsing SYS_ tables on every query would be absurd, so `dict_sys` (`include/dict0dict.h:1133`) caches loaded tables: hash by name, hash by id, an LRU list, all under one `dict_sys->mutex`. The cached objects are the structs every other chapter has been using implicitly:

- `dict_table_t` (`include/dict0mem.h:359`) — name, space id, columns, the list of its indexes, foreign-key lists, lock list.
- `dict_index_t` (`:233`) — **page (root page number)**, space, type (`DICT_CLUSTERED...`), the fields, per-index statistics (`stat_n_diff_key_vals` — fed by random B+tree dives, this era's whole "optimizer statistics"), the `rw_lock_t lock` — yes, *the* tree latch of Chapters 4/6 lives in the dictionary cache entry.

Loading a table by name (`dict_load_table`, `dict/dict0load.c:847`) is plain Chapter 6 machinery: build a search tuple for NAME, descend SYS_TABLES with a persistent cursor, then walk SYS_COLUMNS / SYS_INDEXES / SYS_FIELDS to assemble the object (`:876-1035`).

10.3 A SQL interpreter inside a storage engine?

Surprise: this "no-SQL storage engine" contains a lexer, a yacc grammar, and a query-graph executor (`pars/pars0grm.y`, `pars/pars0lex.l`, `que/que0que.c`, `eval/`). It exists because InnoDB's *own* maintenance operations — creating dictionary rows, foreign-key cascades, purge traversals — are expressed as little SQL procedures executed by this interpreter (`pars_sql()` → `que_run_threads()`, `que/que0que.c:1390` is the canonical entry — `que_eval_sql()`).

DDL shows the pattern (`dict/dict0crea.c`): `CREATE TABLE` builds a graph whose nodes insert rows into SYS_TABLES/SYS_COLUMNS (`tab_create_graph_create`, `:867`), and `CREATE INDEX` inserts into SYS_INDEXES, then calls `btr_create()` and *updates PAGE_NO in the SYS_INDEXES row* (`dict_create_index_tree_step`, `:610`). Schema changes are therefore **transactional row operations on the dictionary tables** — they write undo, they take locks, they can roll back on crash (which is exactly what recovery does with half-finished DDL, Chapter 5's undo phase).

The executor model (`que0que.c:61-80`) — fork/thread/step nodes pumped by `que_run_threads` — is also the scaffolding under every row operation of Chapter 9 (`row_ins_step`, `row_upd_step...`): the embedded API drives the same graph machinery, just with graphs built directly in C rather than parsed from text.

10.4 What to remember

1. The dictionary bootstraps from one fixed page (7) holding root page numbers of four hard-coded system tables; everything else about a schema is ordinary B+tree rows.
2. `dict_index_t.page` is the bridge from names to trees; `dict_index_t.lock` is the tree latch — the dictionary cache is load-bearing for concurrency, not just metadata.

3. DDL = transactional DML on SYS_ tables, executed by InnoDB's private SQL interpreter — crash-safe schema changes fall out of the same undo/redo machinery as everything else.
4. When MySQL 8.0 finally moved *its* data dictionary into InnoDB tables (2018), it was re-adopting the design you're looking at here from ~2000.

Try it: `tests/.libs/ib_ddl` creates and drops tables; break in `dict_create_index_tree_step` to watch a root page get allocated and recorded.

Previous: [Chapter 9 — Row Operations](#) · **Next:** [Chapter 11 — Startup, Shutdown & the Embedded API](#)

Chapter 11 — Startup, Shutdown & the Embedded API

What happens between `ib_init()` and a usable database — and how each `ib_*` call maps to the internals you now know. Source: `api/api0api.c`, `api/api0cfg.c`, `srv/srv0start.c`, `srv/srv0srv.c`

11.1 The embedded API in one test program

Every program in `tests/` follows the same skeleton (see `tests/ib_test1.c`):

```
ib_init();                                /* init memory, sync, OS layers */
ib_cfg_set(...);                         /* configure before startup */
ib_startup("barracuda");                  /* THE big one: open/create/recover */

ib_trx_t trx = ib_trx_begin(IB_TRX_REPEATABLE_READ);
ib_csr_t csr;
ib_cursor_open_table("db/t", trx, &csr);
/* ... ib_cursor_insert_row / ib_cursor_read_row / ib_cursor_next ... */
ib_cursor_close(csr);
ib_trx_commit(trx);

ib_shutdown(IB_SHUTDOWN_NORMAL);
```

11.2 What `ib_startup()` actually does

`ib_startup` funnels into `innobase_start_or_create()` (`srv/srv0start.c:1174`) — the orchestration point where every subsystem from Chapters 1-10 comes online, in dependency order:

1. init core services
memory pool, sync
primitives, fil_system,
buffer pool sized by
buffer_pool_size



2. lock_sys_create
(srv0start.c:1453)
+ spawn I/O handler
threads (:1457)
(simulated async I/O, Ch. 1)



3.
open_or_create_data_files

(:1463)

ibdata* per data_file_path;
if NEW: create space 0,
TRX_SYS page,
doublewrite buffer,
dictionary (dict_create)



4. open/create log files,
log_group_init



5. RECOVERY (Ch. 5)

recv_recovery_from_checkpoint_start
(:1645)

redo apply → dict_boot → trx_sys_init
→ ..._finish (:1671) → rollback of

uncommitted trxs in background
(:1694)

```
graph TD; A[uncommitted trxs in background (:1694)] --> B[6. spawn background threads  
lock-timeout, error-monitor,  
monitor (:1746-1754)  
+ srv_master_thread (:1777) — Ch. 12]; B --> C[engine ready:  
ib_trx_begin / cursors work];
```

6. spawn background
threads
lock-timeout, error-monitor,
monitor (:1746-1754)
+ srv_master_thread
(:1777) — Ch. 12

engine ready:
ib_trx_begin / cursors work

The elegant part is step 5: **there is no "clean start" path vs "crash recovery" path** — the same recovery code always runs; after a clean shutdown it just finds a checkpoint equal to the log end and does nothing. Crash safety isn't a feature bolted on; it *is* the startup path.

Shutdown (`ib_shutdown` → `logs_empty_and_mark_files_at_shutdown`) is recovery's mirror: stop background work, purge/merge what's pending, flush everything, checkpoint at the log end — so the next startup's recovery finds nothing to do.

11.3 Configuration: the `srv_*` globals

`ib_cfg_set("name", value)` writes through a table in `api/api0cfg.c` (:649-1003) directly into the engine's global variables — the same globals MySQL exposes with an `innodb_` prefix:

API name	global	what it tunes (chapter)
<code>buffer_pool_size</code>	<code>srv_buf_pool_size</code>	buffer pool bytes (3)
<code>flush_log_at_trx_commit</code>	<code>srv_flush_log_at_trx_commit</code>	commit durability (5, 7)
<code>log_file_size</code> , <code>log_buffer_size</code>	<code>srv_log_file_size...</code>	redo capacity → checkpoint pressure (5)
<code>data_file_path</code>	<code>srv_data_file_names...</code>	system tablespace layout (1)
<code>file_per_table</code>	<code>srv_file_per_table</code>	one .ibd tablespace per table (1)
<code>lru_old_blocks_pct</code> , <code>lru_block_access_recency</code>	<code>buf_LRU_old_*</code>	midpoint LRU (3)
<code>max_dirty_pages_pct</code>	<code>srv_max_buf_pool_modified_pct</code>	flush pressure (12)
<code>max_purge_lag</code>	<code>srv_max_purge_lag</code>	purge backlog throttle (7)
<code>adaptive_hash_index</code>	<code>btr_search_enabled</code>	AHI (6)
<code>lock_wait_timeout</code>	<code>ses_lock_wait_timeout</code>	lock waits (8)

Reading this table is a Rosetta stone: it maps two decades of MySQL tuning folklore onto the specific mechanisms in this codebase.

11.4 API call → internal machinery

The `api/` layer is deliberately thin — argument marshalling plus calls into `row/`:

API call	what happens inside
<code>ib_trx_begin(level)</code>	<code>trx_allocate_for_client</code> + <code>trx_start</code> (<code>api0api.c:856-866</code>) — id assigned, rseg chosen (Ch. 7)
<code>ib_cursor_open_table</code>	dictionary lookup (<code>dict_table_get</code> , Ch. 10) + build <code>row_prebuilt_t</code> with a persistent cursor (<code>:3089-3131</code>)
<code>ib_cursor_insert_row</code>	build/reuse an insert query graph, pump <code>row_ins_step</code> with lock-wait retry (<code>:3353-3420</code> , Ch. 9.2)
<code>ib_cursor_moveto</code> / <code>read_row</code> / <code>next</code>	<code>row_search_for_client</code> (<code>:3964-4090</code> , Ch. 9.3)
<code>ib_cursor_update_row</code> / <code>delete_row</code>	<code>row_upd_step</code> graphs (<code>:3568-3687</code> , Ch. 9.4)
<code>ib_trx_commit</code>	<code>trx_commit</code> (Ch. 7.4) — undo state flip + lock release + group-commit log flush
<code>ib_cursor_lock</code> / <code>ib_table_lock</code>	<code>lock_table</code> (Ch. 8)

One pattern deserves attention: **lock-wait retry loops** (`ib_insert_row_with_lock_retry`, `api0api.c:3226`). When the row layer returns `DB_LOCK_WAIT`, the API layer suspends the thread until the lock is granted or times out, then *re-runs the operation from the stored cursor position*. In MySQL this loop lives in the handler layer; here you can read it in isolation — persistent cursors (Chapter 6) are what make the retry possible.

11.5 What to remember

1. Startup is a strict dependency ladder: OS/sync → buffer pool + I/O threads → files → **recovery always** → background threads. Shutdown arranges for recovery to find nothing.
2. Configuration = writing `srv_*` globals before startup; the API names map 1:1 onto `innodb_*` variables you already know from MySQL.
3. The API layer adds no semantics — it builds prebuilt structs + query graphs and handles `DB_LOCK_WAIT` retries; all real work is Chapters 6-10.

Try it: `ltrace -e 'ib_*' tests/.libs/ib_test1 2>&1 | head -50` shows the startup and transaction ceremony in order; `tests/README` and `docs/api-reference.md` list the full API.

Previous: Chapter 10 — The Data Dictionary · **Next:** Chapter 12 — Background Threads

Chapter 12 — Background Threads, the Master Thread & the Insert Buffer

Who does the work your transaction didn't wait for — deferred flushing, purging, merging, checkpointing. Source: `srv/srv0srv.c`, `srv/srv0start.c`, `ibuf/ibuf0ibuf.c`

12.1 The thread roster

A running engine has surprisingly few threads (`srv0start.c:1457-1777`):

thread	role
<code>N × io_handler_thread (:593)</code>	complete simulated-async I/O requests (Ch. 1) — segments for log, ibuf, reads, writes
<code>srv_master_thread (srv0srv.c:2215)</code>	<i>the</i> background worker — below
<code>srv_lock_timeout_thread (srv0srv.c:1961)</code>	wakes lock waiters whose timeout expired (Ch. 8)
<code>srv_error_monitor_thread (:2059)</code>	detects long semaphore waits (latch bugs/stalls, Ch. 4)
<code>srv_monitor_thread (:1754)</code>	prints the InnoDB monitor output
user threads	your application's threads executing <code>ib_*</code> calls

Note what's *not* here: no dedicated purge threads, no page-cleaner threads, no log-writer thread — all of that lives inside **one master thread** in this era. Watching later InnoDB versions peel these responsibilities into dedicated threads is watching the scalability story of the 2010s.

12.2 The master thread: the engine's heartbeat

`srv_master_thread()` (`srv/srv0srv.c:2215`) is a loop with two cadences — a translation of the classic "spend spare I/O capacity wisely" policy into code:

```

every second (srv0srv.c:2280-2384):
• flush the log buffer ← makes flush_log_at_trx_commit=0/2 safe-ish
• if I/O was idle: merge insert buffer (5% of I/O capacity)
• if buf pool too dirty (> max_dirty_pages_pct):
    buf_flush_batch(BUF_FLUSH_LIST, 100% of I/O capacity)
• adaptive flushing based on redo production rate

every ten seconds (:2386-2464):
• flush 100 dirty pages if I/O was idle
• ALWAYS merge some insert buffer
• trx_purge() until history drained ← MVCC garbage collection (Ch. 7)
• flush oldest pages (100 if >70% dirty, else 10)
• log_checkpoint() ← bound recovery time (Ch. 5)

when the server is idle: background loop – purge, ibuf merge, big flushes

```

Every decision is gated on measured I/O activity (`PCT_IO(n)` = percent of the configured `io_capacity`, thresholds at `srv0srv.c:406-408`): background work must not steal bandwidth from user transactions, but idle disks should be earning their keep. That single idea — *deferred work* + *idle-time scheduling* — is why InnoDB commits fast: a transaction only pays for redo-log durability at commit; page writes, purging, ibuf merging, and checkpointing are all somebody else's (this thread's) problem.

12.3 The insert buffer: turning random I/O into sequential

The last major structure in the engine, and this era's signature optimization (`ibuf/ibuf0ibuf.c`).

Problem. Every row INSERT updates each secondary index. Secondary-index keys are essentially random (that's why you built the index), so each insert wants a *random leaf page* — if it's not cached, that's a random disk read *per index per row*. Bulk loads die by this.

Solution. If the target leaf page of a **non-unique secondary index** is not in the buffer pool (`btr_cur_search_to_nth_level` probes with `BUF_GET_IF_IN_POOL`, `btr/btr0cur.c:537-544`), don't read it. Instead, insert the entry into the **insert buffer tree** — a system B+tree (space 0, root at page 4, Ch. 1) keyed by (space, page_no, record) — and mark completion (`ibuf_insert`, `ibuf/ibuf0ibuf.c:2842`). The random read is avoided entirely.

Merging. Buffered entries are applied when the page comes into the pool anyway:

- on a normal read of the page (`ibuf_merge_or_delete_for_page`, `:3150`), or
- proactively by the master thread (`ibuf_contract_for_n_pages`, `:2282`), which reads target pages in batches (turning many random writes into few clustered ones).

Bookkeeping. How do you know a page has pending entries without reading it? The **ibuf bitmap pages** (Ch. 1: page 1 of every 16384-page block): 4 bits per page — free-space estimate, a "has buffered entries" bit, and an "is an ibuf page" bit

(`IBUF_BITMAP_FREE/BUFFERED/IBUF`, `ibuf0ibuf.c:214-218`). The bitmap is consulted on every buffered insert and cleared on merge.

The fine print — why only *non-unique* secondary indexes: a UNIQUE check must read the actual leaf page to detect duplicates, which defeats the whole point; and clustered-index pages are where the row itself goes, so they're needed anyway (asserted at `ibuf0ibuf.c:2858`). There's also a subtle crash-safety story: `ibuf` entries and their merges are all redo-logged B+tree operations, so recovery replays them like everything else — but recovery must *not* trigger new merges while the redo scan runs (`recv_no_ibuf_operations`, Ch. 5).

Modern MySQL renamed this the "change buffer" (buffering deletes and purges too) — and MySQL 8.4 finally disabled it by default in 2024, because SSDs made random reads cheap. You are reading the code of an optimization across its entire life cycle: invented for spinning disks, obsoleted by flash.

12.4 The full picture

With the insert buffer, the engine diagram from Chapter 0 is complete. One last mental replay — a single-row INSERT touches, in order:

```
api (Ch.11) → row_ins (Ch.9) → dict lookup (Ch.10) → btr descent (Ch.6)
  → [leaf cached? else ibuf (Ch.12)] → page_cur insert (Ch.2)
  → inside an mtr: latches (Ch.4) + redo (Ch.5) → undo record (Ch.7)
  → implicit lock (Ch.8) → dirty page in buffer pool (Ch.3)
commit: undo state flip + log group-commit (Ch.7, 5)
later, invisibly: master thread flushes the page via doublewrite (Ch.3, 12),
  checkpoint advances (Ch.5), purge/ibuf-merge tidy up (Ch.7, 12)
crash at ANY point: recovery redoes, then undoes (Ch.5) — no step above is lost.
```

If you can narrate that chain from memory, you understand how InnoDB was architected.

12.5 What to remember

1. One master thread runs the engine's metabolism on 1s/10s cadences, spending only idle I/O — deferred work is the reason commits are fast.
2. The insert buffer trades "random read now" for "sequential merge later", tracked by bitmap pages; it's the era's boldest optimization and a lesson in hardware-driven design.
3. Timeouts, monitors, and I/O completion each get a small dedicated thread; everything else is done by whoever's transaction needs it — a design later versions had to unbundle to scale on many-core machines.

Try it: run `tests/run_bulk_test.sh 100000 1000 4`, then break in `srv_master_thread` — step through one 10-second cycle and watch it call `ibuf_contract_for_n_pages`, `trx_purge`, `buf_flush_batch`, and `log_checkpoint` in turn.

Previous: [Chapter 11 — Startup & API](#) · **Back to:** [Chapter 0 — Overview](#)